🧠 AI Evolution Program

Parte 00 — Fundamentos, historia y método científico

Construye el mapa del campo, su historia, el vocabulario técnico y el método experimental que se utilizará en todo el programa.

1. 001 — Qué es inteligencia artificial y qué no es
2. 002 — De Turing a Dartmouth: nacimiento formal del campo
3. 003 — Inviernos, resurgimientos y ciclos de expectativas
4. 004 — Agentes racionales, entornos y medidas de desempeño
5. 005 — Vectores, matrices y geometría para IA
6. 006 — Probabilidad, incertidumbre y estadística básica
7. 007 — Lógica, algoritmos y complejidad computacional
8. 008 — Datos, evidencia, hipótesis y falsabilidad
9. 009 — Entornos Python, Git y experimentos reproducibles
10. 010 — Cómo leer papers, benchmarks y claims de IA
11. 011 — Ética desde el diseño y límites de automatización
12. 012 — Proyecto: mapa evolutivo verificable de la IA

001 — Qué es inteligencia artificial y qué no es

Parte: 00 — Fundamentos, historia y método científico
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `frontier` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **qué es inteligencia artificial y qué no es** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar qué es inteligencia artificial y qué no es usando los conceptos `IA estrecha`, `IA general`, `automatización`, `autonomía`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`IA estrecha`, `IA general`, `automatización`, `autonomía`

🗺️ Ubicación en el mapa de la IA

Esta clase es el punto cero del programa: antes de estudiar algoritmos concretos hay que acotar el objeto de estudio. La definición operativa de IA que se adopte aquí determina cómo se leerán los hitos históricos (clase 002), los ciclos de expectativas (clase 003) y el marco de agentes racionales (clase 004). Sin esta demarcación, cualquier sistema con un `if` puede venderse como "inteligente" y cualquier promesa de "IA general" puede pasar sin escrutinio.

📖 Fundamentos

🎯 Cuatro definiciones clásicas de IA

Russell y Norvig (AIMA 4e, cap. 1) organizan las definiciones históricas de IA en una matriz de dos ejes: **pensar vs. actuar** y **como humanos vs. racionalmente**:

	Como humanos	Racionalmente
Pensar	Ciencia cognitiva: modelar la mente humana	Leyes del pensamiento: lógica formal
Actuar	Test de Turing: conducta indistinguible	Agente racional: maximizar una medida de desempeño

El enfoque dominante hoy es el del **agente racional**: un sistema que percibe su entorno y actúa para maximizar el valor esperado de una medida de desempeño, dadas las percepciones disponibles y su conocimiento previo. Esta definición es *operativa*: no exige conciencia ni "entendimiento", exige desempeño medible.

🔬 IA estrecha vs. IA general

- **IA estrecha (narrow AI / weak AI):** sistemas competentes en una tarea o dominio acotado (clasificar imágenes, traducir, jugar Go, completar texto). Todo sistema desplegado comercialmente hasta hoy pertenece a esta categoría. Su competencia **no transfiere** automáticamente fuera de la distribución de datos para la que fue construido.
- **IA general (AGI):** sistema hipotético con competencia comparable a la humana en la mayoría de tareas cognitivas económicamente relevantes, incluyendo transferencia entre dominios y aprendizaje con pocos datos. Es un objetivo de investigación, no un artefacto existente; cualquier afirmación de que un sistema actual "es AGI" debe tratarse como un claim extraordinario que exige evidencia extraordinaria.

Un error de categoría frecuente: la fluidez lingüística de un modelo generativo se percibe como generalidad. Fluidez ≠ generalidad: la competencia debe evaluarse por tarea, con distribuciones de prueba distintas a las de entrenamiento.

⚙️ Automatización vs. autonomía

Dos conceptos que el marketing mezcla y la ingeniería debe separar:

- **Automatización:** ejecutar sin intervención humana un procedimiento *especificado de antemano*. Una macro, un cron job o un pipeline ETL son automatización sin IA. El comportamiento es trazable a reglas escritas por personas.
- **Autonomía:** capacidad de un sistema para *seleccionar sus propias acciones* ante situaciones no enumeradas explícitamente por el diseñador, usando percepción y algún criterio de decisión. La autonomía admite grados (los niveles de conducción autónoma SAE 0-5 son el ejemplo canónico) y siempre está acotada por el dominio de operación diseñado (ODD, *operational design domain*).

Una prueba práctica de demarcación en tres preguntas:

```
1. ¿El mapeo entrada→salida fue escrito a mano?        → automatización clásica
2. ¿El mapeo se indujo desde datos u optimización?     → aprendizaje automático (IA estrecha)
3. ¿El sistema decide qué acción tomar en situaciones
   no enumeradas, bajo una medida de desempeño?        → agente con autonomía (acotada)
```

🚫 Qué NO es IA (hoy)

- No es magia estadística sin supuestos: todo modelo hereda los sesgos y la cobertura de sus datos.
- No es conciencia ni intencionalidad: optimizar una función de pérdida no implica "querer".

- No es infalibilidad: los sistemas de IA fallan de formas distintas (y a veces más silenciosas) que el software clásico, porque su especificación es implícita en los datos.
- No es un sustituto de la especificación del problema: si la medida de desempeño está mal elegida, el sistema optimizará lo incorrecto con gran eficacia (ley de Goodhart).

🧮 Ejemplo trabajado

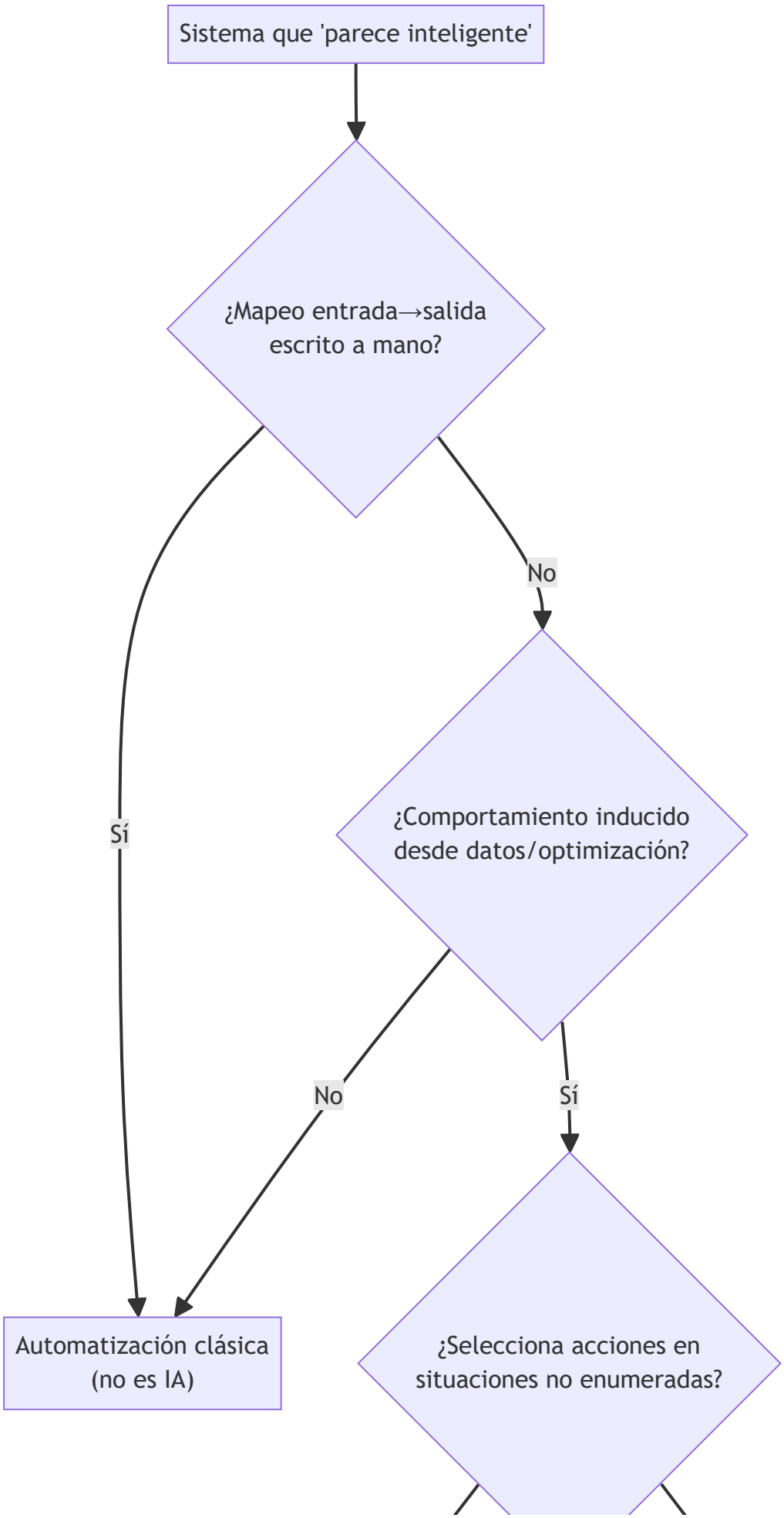
Clasifiquemos cuatro sistemas con la prueba de demarcación:

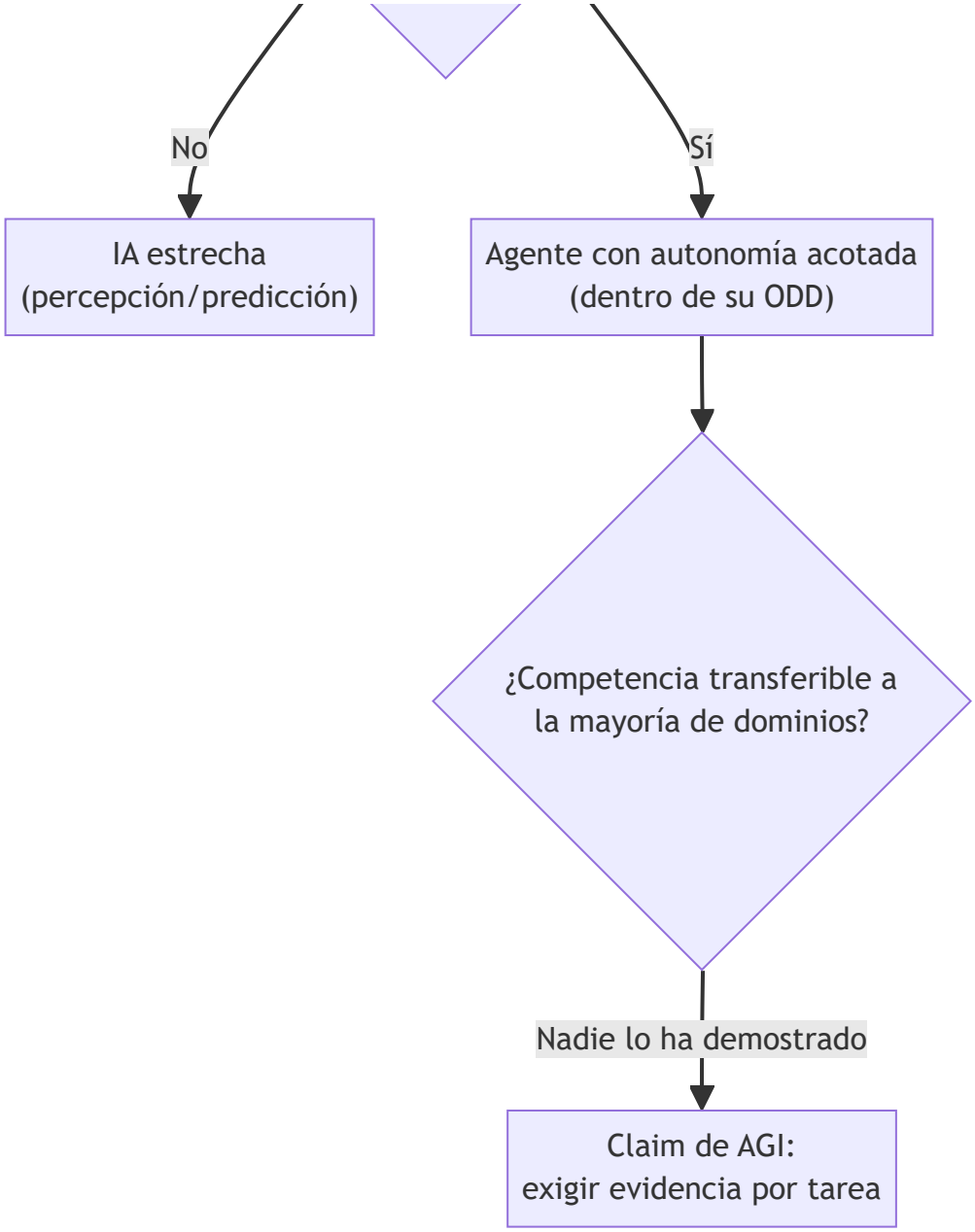
Sistema	¿Reglas a mano?	¿Inducido de datos?	¿Decide acciones?	Veredicto
Termostato on/off a 22 °C	Sí (umbral fijo)	No	No	Automatización, no IA
Filtro de spam bayesiano	No	Sí (frecuencias de palabras)	No (solo etiqueta)	IA estrecha (clasificador)
Robot aspirador que mapea la casa	Parcial	Sí (SLAM, percepción)	Sí (ruta, dentro de un ODD)	Agente con autonomía acotada
"Asistente AGI" anunciado en una demo	?	?	?	Claim sin evidencia: exigir evaluación por tarea

Traza numérica para el filtro de spam: con un corpus de 1 000 correos (200 spam), la palabra "gratis" aparece en 120 spam y 40 legítimos. P("gratis") = 160/1000 = 0.16 y P("gratis"|spam)·P(spam) = (120/200)·(200/1000) = 0.12. Por Bayes, P(spam|"gratis") = 0.12/0.16 = **0.75**. El sistema no "entiende" el correo: computa frecuencias. Eso es IA estrecha funcionando exactamente como fue diseñada — y también su límite.

📊 Propiedades y comparación

Dimensión	Automatización clásica	IA estrecha	IA general (hipotética)
Origen del comportamiento	Reglas explícitas	Inducción desde datos	Transferencia entre dominios
Fallo típico	Caso no previsto → excepción	Cambio de distribución → error silencioso	— (no existe artefacto)
Auditabilidad	Alta (código legible)	Media-baja (pesos opacos)	Desconocida
Evidencia exigible	Tests unitarios	Evaluación fuera de distribución, baselines	Claim extraordinario
Estado en 2026	Ubicua	Ubicua	Objetivo de investigación


Sistema que 'parece inteligente'
¿Mapeo entrada→salida escrito a mano?
Sí
No
¿Comportamiento inducido desde datos/optimización?
No
Sí
Automatización clásica (no es IA)
¿Selecciona acciones en situaciones no enumeradas?



⚠️ Errores conceptuales frecuentes

1. **"Si usa un modelo grande, es inteligente en general."** La escala mejora el desempeño dentro de la distribución de entrenamiento; la generalidad se demuestra con evaluaciones por tarea fuera de esa distribución, no con fluidez aparente.
2. **"Automatizar con reglas ya es IA."** Un árbol de `if/else` escrito a mano es software determinista clásico; llamarlo IA infla expectativas y confunde la auditoría.
3. **"El sistema decidió solo, nadie es responsable."** La autonomía es siempre delegada y acotada por diseño; la responsabilidad permanece en quien define la medida de desempeño y el dominio de operación.
4. **"Pasar una conversación convincente demuestra pensamiento."** El propio Turing (1950) propuso el juego de imitación como *sustituto operativo* de la pregunta "¿pueden pensar las máquinas?", que consideró demasiado ambigua — no como prueba de conciencia.
5. **"La IA elimina el sesgo humano."** Un modelo entrenado con decisiones humanas históricas reproduce y a veces amplifica esos sesgos, con la agravante de parecer objetivo.

🚀 Del aprendizaje a la operación

Entre esta taxonomía y un sistema real median: la especificación formal del dominio de operación (qué entradas son válidas y cuáles se rechazan), la medición continua del cambio de distribución en producción, un baseline no-IA contra el cual justificar la complejidad añadida, y un protocolo de escalamiento a revisión humana cuando la confianza del sistema cae. Clasificar correctamente el sistema (automatización / IA estrecha / agente) determina qué régimen de pruebas y auditoría le corresponde.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("frontier")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Turing, A. M. (1950). Computing Machinery and Intelligence. *Mind*, LIX(236) — uso: fuente primaria del mecanismo estudiado
- McCarthy, Minsky, Rochester & Shannon (1955). A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence — uso: referencia consultada en su fuente original
- McCarthy, J. What is Artificial Intelligence? — uso: referencia consultada en su fuente original
- Russell, S. & Norvig, P. *Artificial Intelligence: A Modern Approach*, 4.ª ed., cap. 1 — uso: desarrollo extendido del tema
- Nilsson, N. (2010). *The Quest for Artificial Intelligence* (PDF oficial gratuito) — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P54 · Un cálculo lógico de las ideas inmanentes en la actividad nerviosa	1943	Establece que una red de neuronas de umbral puede calcular cualquier función lógica: el puente entre biología y computación.	notebook
P56 · Maquinaria computacional e inteligencia	1950	Cambia una pregunta metafísica —¿pueden pensar las máquinas?— por un procedimiento que se puede ejecutar y discutir.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 1 · obra de referencia de la parte 00

📝 Evaluación completa

❓ Preguntas

1. Define qué es inteligencia artificial y qué no es sin usar una marca o framework como definición.
2. Explica la relación entre IA estrecha, IA general, automatización, autonomía.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

002 — De Turing a Dartmouth: nacimiento formal del campo

Parte: 00 — Fundamentos, historia y método científico
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `frontier` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **de turing a dartmouth: nacimiento formal del campo** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar de turing a dartmouth: nacimiento formal del campo usando los conceptos `Turing`, `Dartmouth`, `historia`, `hitos`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`Turing`, `Dartmouth`, `historia`, `hitos`

🗺️ Ubicación en el mapa de la IA

Entre 1936 y 1956 se ensamblan las piezas que convierten "máquinas que piensan" de especulación filosófica en programa de investigación: computabilidad (Turing 1936), neuronas formales (McCulloch y Pitts 1943), un criterio operativo de inteligencia (Turing 1950) y un nombre con agenda propia (Dartmouth 1956). Todo lo que sigue en el programa — desde búsqueda hasta modelos de lenguaje — desciende de decisiones tomadas en esta ventana de veinte años.

📖 Fundamentos

🧱 1936: la máquina universal

En *On Computable Numbers* (1936), Turing formaliza qué significa "computar": una máquina abstracta con cinta infinita, cabezal y tabla finita de estados. Dos resultados fundan el campo: existe una **máquina universal** capaz de simular cualquier otra máquina si se le da su descripción (el fundamento conceptual del software y del programa almacenado), y el **Entscheidungsproblem** es indecidible: hay preguntas bien

definidas que ningún algoritmo puede responder para todos los casos. La IA nace, por tanto, dentro de límites demostrados, no en un espacio de posibilidad infinita.

🧠 1943: la neurona formal

McCulloch y Pitts proponen la primera abstracción matemática de la neurona: unidades binarias con umbral cuyas redes pueden computar cualquier función lógica proposicional.

```
salida = 1  si  Σ (entradas excitatorias) ≥ umbral  y  ninguna entrada inhibitoria activa
salida = 0  en caso contrario
```

Es el ancestro directo del perceptrón (1958) y, con activaciones continuas y pesos ajustables, de las redes profundas actuales. Su aporte no fue biológico sino formal: demostró que "lo mental" podía describirse como cómputo.

🎭 1950: el juego de imitación

En *Computing Machinery and Intelligence* (Mind, 1950), Turing reemplaza la pregunta "¿pueden pensar las máquinas?" — que juzga "demasiado carente de sentido para merecer discusión" en su forma original — por un experimento operativo: un interrogador conversa por teletipo con un humano y una máquina; si no puede distinguirlos de forma fiable, la máquina pasa la prueba. El artículo además enumera y refuta nueve objeciones (teológica, matemática vía Gödel, de la conciencia, de Lady Lovelace, etc.) y cierra proponiendo **máquinas que aprenden** como camino práctico — anticipando el aprendizaje automático por décadas.

🏛️ 1956: Dartmouth y el nombre del campo

La propuesta de McCarthy, Minsky, Rochester y Shannon (1955) para el taller de verano de Dartmouth formula la **conjetura fundacional**: "cada aspecto del aprendizaje o cualquier otra característica de la inteligencia puede, en principio, describirse con tanta precisión que una máquina puede simularlo". El documento fija la agenda: lenguaje, redes neuronales, abstracción, creatividad y automejora. McCarthy acuña ahí el término *artificial intelligence*, en parte para distinguir el programa de la cibernética de Wiener. El taller no resolvió nada por sí mismo, pero creó la comunidad: Newell y Simon presentaron el Logic Theorist, considerado el primer programa de IA operativo.

📈 1949-1959: el aprendizaje entra en escena

Dos hitos completan el nacimiento del campo: la regla de Hebb (1949, "las neuronas que se disparan juntas se conectan") da un mecanismo de aprendizaje plausible, y el programa de damas de Samuel (1959) demuestra empíricamente que una máquina puede **superar a su programador** aprendiendo de la experiencia — introduciendo términos como *machine learning* y técnicas precursoras del aprendizaje por refuerzo.

🧮 Ejemplo trabajado

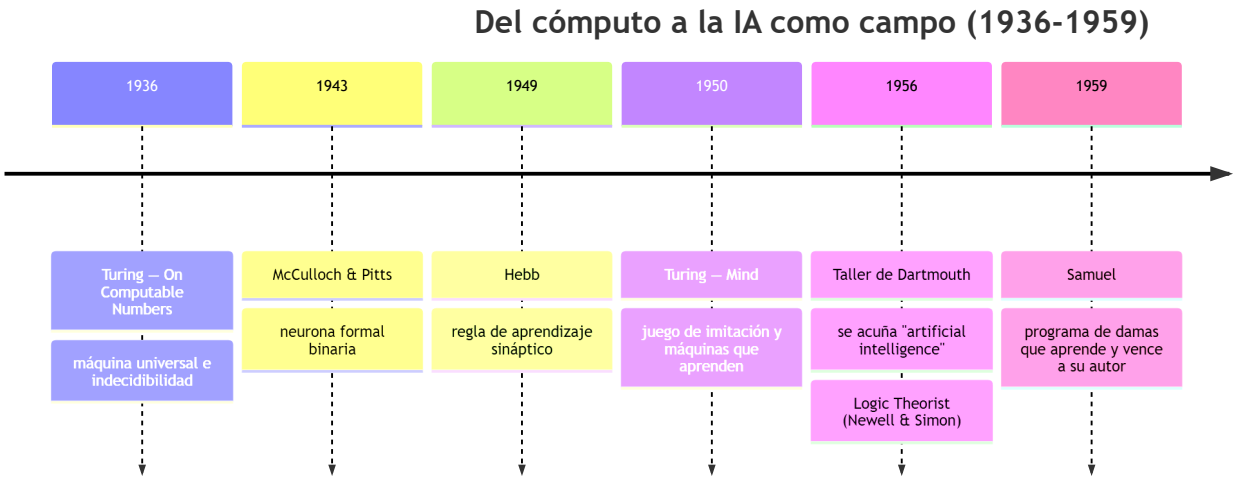
Verifiquemos que una neurona de McCulloch-Pitts computa la función AND con umbral $\theta = 2$ y pesos excitatorios $w_1 = w_2 = 1$:

x_1	x_2	$\Sigma = w_1x_1 + w_2x_2$	$\Sigma \geq 2$?	salida
0	0	0	no	0
0	1	1	no	0
1	0	1	no	0
1	1	2	sí	1

Con $\theta = 1$ la misma unidad computa OR; añadiendo una entrada inhibitoria se obtiene NOT. Como {AND, OR, NOT} es funcionalmente completo, redes de estas unidades computan cualquier función booleana — exactamente el resultado de 1943. Lo que la unidad NO puede hacer sola es XOR (no es linealmente separable), la limitación que Minsky y Papert explotarían en 1969 contra el perceptrón.

📊 Propiedades y comparación

Hito	Año	Aporte	Qué NO resolvió
Turing, máquina universal	1936	Define computabilidad; programa almacenado	Nada sobre inteligencia práctica
McCulloch & Pitts	1943	Neurona formal; mente como cómputo	Sin regla de aprendizaje
Hebb	1949	Mecanismo de aprendizaje sináptico	Sin formalización algorítmica
Turing, juego de imitación	1950	Criterio operativo de inteligencia	Mide indistinguibilidad, no competencia
Dartmouth	1956	Nombre, agenda y comunidad	Subestimó radicalmente la dificultad
Samuel, damas	1959	Aprendizaje que supera al programador	Dominio cerrado y minúsculo



⚠️ Errores conceptuales frecuentes

1. **"El test de Turing es la definición oficial de IA."** Es un criterio operativo de 1950, hoy más histórico que práctico: mide indistinguibilidad conversacional, no competencia general, y es vulnerable

a trucos de imitación superficial (chatbots tipo ELIZA).
2. **"Dartmouth fue donde se inventó la IA."** Los resultados técnicos clave (computabilidad, neurona formal, Logic Theorist) son anteriores o simultáneos; Dartmouth aportó el nombre, la agenda y la comunidad.
3. **"Turing propuso el test porque creía que las máquinas piensan como humanos."** Propuso *reemplazar* esa pregunta por una decidible experimentalmente; su posición sobre la pregunta original fue deliberadamente agnóstica.
4. **"Las redes neuronales son una idea reciente."** La neurona formal es de 1943; lo reciente es la combinación de datos masivos, GPU y retropropagación a escala.
5. **"La conjetura de Dartmouth quedó demostrada."** Sigue siendo una conjetura: que la inteligencia sea describible con precisión suficiente para simularse es la apuesta del campo, no un teorema.

🚀 Del aprendizaje a la operación

Leer las fuentes primarias inmuniza contra dos vicios profesionales: atribuir a los sistemas actuales propiedades que nadie ha demostrado (la conjetura de Dartmouth tratada como hecho) y descartar límites formales conocidos (indecidibilidad, separabilidad lineal) al estimar qué puede automatizarse. En un proyecto real, este bagaje se traduce en exigir criterios operativos de éxito — el gesto de Turing de 1950 — antes de aceptar afirmaciones sobre "inteligencia" de un producto.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("frontier")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Turing, A. M. (1950). Computing Machinery and Intelligence. *Mind*, LIX(236), 433-460 — uso: fuente primaria del mecanismo estudiado
- Turing, A. M. (1936). On Computable Numbers, with an Application to the Entscheidungsproblem — uso: fuente primaria del mecanismo estudiado
- McCarthy, Minsky, Rochester & Shannon (1955). A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence — uso: referencia consultada en su fuente original
- McCulloch, W. & Pitts, W. (1943). A Logical Calculus of the Ideas Immanent in Nervous Activity — uso: fuente primaria del mecanismo estudiado
- Samuel, A. (1959). Some Studies in Machine Learning Using the Game of Checkers. *IBM Journal* — uso: fuente primaria del mecanismo estudiado

- Russell, S. & Norvig, P. *AIMA*, 4.ª ed., cap. 1 (historia del campo) — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P56 · Maquinaria computacional e inteligencia	1950	Cambia una pregunta metafísica —¿pueden pensar las máquinas?— por un procedimiento que se puede ejecutar y discutir.	notebook
P57 · Propuesta para el proyecto de investigación de verano de Dartmouth sobre inteligencia artificial	1955	Bautiza el campo y fija su agenda: siete temas que aún organizan buena parte de la investigación.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 1 · obra de referencia de la parte 00

📝 Evaluación completa

❓ Preguntas

1. Define de turing a dartmouth: nacimiento formal del campo sin usar una marca o framework como definición.
2. Explica la relación entre Turing, Dartmouth, historia, hitos.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.

5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

003 — Inviernos, resurgimientos y ciclos de expectativas

Parte: 00 — Fundamentos, historia y método científico
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `evaluation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **inviernos, resurgimientos y ciclos de expectativas** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar inviernos, resurgimientos y ciclos de expectativas usando los conceptos `AI winters`, `expectativas`, `evidencia`, `mercado`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`AI winters`, `expectativas`, `evidencia`, `mercado`

🗺️ Ubicación en el mapa de la IA

Tras el optimismo fundacional de Dartmouth (clase 002), el campo vivió ciclos de euforia, promesa incumplida y recorte de financiación conocidos como **inviernos de la IA**. Estudiar estos ciclos no es arqueología: da el instrumental para evaluar los claims del presente — incluida la ola actual de IA generativa — y conecta directamente con la lectura crítica de papers y benchmarks (clase 010).

📖 Fundamentos

📉 Anatomía de un invierno

Un invierno de IA es un periodo de colapso de financiación y credibilidad tras expectativas infladas. El patrón se repite con estructura reconocible:

```
1. Avance técnico real y demostrable en un dominio acotado
2. Extrapolación pública: "en N años, inteligencia humana"
3. Financiación masiva atada a la extrapolación, no al avance
4. La dificultad real aparece (explosión combinatoria, falta de datos/cómputo)
5. Informe o fracaso visible que cristaliza la decepción
6. Recorte de fondos, éxodo de talento, cambio de etiqueta ("informática avanzada")
```

❄️ Primer invierno (~1974-1980)

Detonantes documentados:

- **Informe ALPAC (1966, EE. UU.):** concluyó que la traducción automática era más cara y peor que la humana; cortó la financiación de MT durante una década.
- **Minsky y Papert, *Perceptrons* (1969):** demostraron formalmente que el perceptrón de una capa no puede computar funciones no linealmente separables (XOR); el resultado, junto a su influencia institucional, congeló la investigación en redes neuronales.
- **Informe Lighthill (1973, Reino Unido):** encargado por el Science Research Council, concluyó que la IA no había cumplido sus promesas y que la **explosión combinatoria** hacía inviables los métodos de búsqueda general fuera de problemas de juguete. El parlamento británico desmanteló la financiación salvo en tres universidades.

La causa técnica de fondo: los métodos "débiles" (búsqueda general + heurísticas) escalan exponencialmente con el tamaño del problema, y el hardware de la época agravaba el muro.

🏭 Auge y segundo invierno (~1980-1993)

Los **sistemas expertos** (MYCIN, XCON/R1) codificaban reglas de especialistas y produjeron valor real: XCON ahorraba a DEC decenas de millones de dólares anuales configurando computadores VAX. Japón lanzó el proyecto **Quinta Generación** (1982) y EE. UU. respondió con DARPA/SCI. El colapso (~1987-1993) llegó por: costo de mantenimiento de bases de reglas frágiles (el "cuello de botella de adquisición de conocimiento"), incapacidad de manejar incertidumbre y casos fuera de las reglas, y el desplome del mercado de máquinas Lisp ante estaciones de trabajo genéricas más baratas.

🔁 El resurgimiento y el ciclo de expectativas

Desde ~1995 el campo se reconstruyó sobre bases más sólidas: métodos probabilísticos (redes bayesianas), aprendizaje estadístico con evaluación en benchmarks, y desde 2012 (AlexNet en ImageNet) el aprendizaje profundo habilitado por GPU y datos masivos. El modelo descriptivo popular para estos ciclos es el *hype cycle* de Gartner (pico de expectativas infladas → abismo de desilusión → pendiente de iluminación → meseta de productividad). Es útil como vocabulario, con una advertencia: es un esquema de marketing, no una ley empírica validada — usarlo como si predijera fechas es repetir el error que describe.

Indicadores para distinguir avance real de burbuja:

- ¿La métrica mejora en benchmarks *independientes* y fuera de la distribución de entrenamiento?
- ¿El costo por unidad de tarea baja de forma sostenida?
- ¿Hay despliegues en producción con usuarios que pagan, o solo demos controladas?
- ¿Las promesas tienen fecha y criterio de éxito falsable?

Ejemplo trabajado

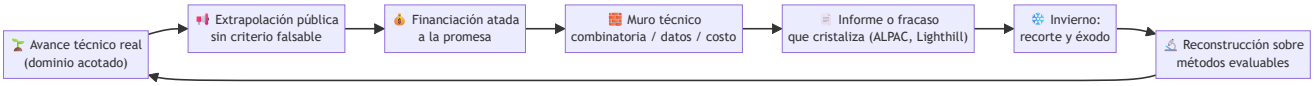
Apliquemos el patrón de 6 pasos al primer invierno, con fechas verificables:

Paso del patrón	Evento histórico	Año
1. Avance real	Logic Theorist, SHRDLU en micromundos	1956-1972
2. Extrapolación	Simon: "en 20 años las máquinas harán cualquier trabajo humano"	1965
3. Financiación atada a la promesa	DARPA financia traducción automática y IA general	1963-1973
4. Dificultad real	Explosión combinatoria fuera de problemas de juguete	~1970
5. Cristalización	ALPAC (1966), *Perceptrons* (1969), Lighthill (1973)	1966-1973
6. Recorte	DARPA y SRC cortan fondos; el término "IA" se vuelve tóxico	1974-1980

Ejercicio de lectura: la afirmación de Simon (paso 2) era falsable — tenía plazo y alcance — y resultó falsa. Eso la hace *mejor ciencia* que una promesa vaga sin plazo, pero peor gestión de expectativas. La lección no es "no predecir", sino auditar predicciones pasadas antes de aceptar nuevas del mismo emisor.

Propiedades y comparación

Dimensión	1.er invierno (1974-80)	2.º invierno (1987-93)	¿Hoy? (a auditar)
Tecnología estrella	Búsqueda + heurísticas	Sistemas expertos	Modelos generativos
Promesa inflada	Inteligencia general en ~20 años	Conocimiento experto barato	AGI inminente
Muro técnico	Explosión combinatoria	Adquisición/fragilidad de reglas	Costo, alucinación, evaluación
Detonante visible	ALPAC, Perceptrons, Lighthill	Colapso de máquinas Lisp	(sin determinar)
Qué sobrevivió	Algoritmos de búsqueda, teoría	Motores de reglas, ontologías	(sin determinar)



Errores conceptuales frecuentes

1. **"Los inviernos prueban que la IA no funciona."** Lo que colapsó fueron expectativas infladas; los métodos con evidencia sólida (búsqueda, reglas, luego estadística) sobrevivieron y se integraron al software ordinario.
2. **"Esta vez es diferente, no puede haber otro invierno."** Puede que sí o que no: la pregunta correcta es qué indicadores falsables distinguen esta ola de las anteriores, no la confianza subjetiva de los participantes.
3. **"El informe Lighthill era anti-ciencia."** Fue un análisis técnico encargado por el financiador que identificó correctamente la explosión combinatoria; su error estuvo en generalizar el diagnóstico a todo el campo.
4. **"El hype cycle de Gartner es una ley."** Es una heurística descriptiva sin validación empírica sistemática; muchas tecnologías nunca recorren la curva completa.

5. **"Los sistemas expertos fracasaron por ser simbólicos."** Fracasaron por costo de mantenimiento y fragilidad ante incertidumbre; la representación simbólica sigue viva en motores de reglas, planificación y verificación.

🚀 Del aprendizaje a la operación

En un contexto profesional este tema se convierte en gestión de riesgo tecnológico: antes de apostar presupuesto a una capacidad de IA, exigir evidencia reproducible en el dominio propio (no en la demo del vendedor), separar la mejora medible de la narrativa, fijar criterios de salida si la promesa no se cumple en plazo, y presupuestar el mantenimiento — la causa silenciosa del segundo invierno — tanto como el desarrollo inicial.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("evaluation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Lighthill, J. (1973). Artificial Intelligence: A General Survey (informe al Science Research Council) — uso: referencia consultada en su fuente original
- Russell, S. & Norvig, P. *AIMA*, 4.ª ed., cap. 1.3 (historia de la IA) — uso: desarrollo extendido del tema
- Nilsson, N. (2010). *The Quest for Artificial Intelligence*, caps. sobre auge y colapso (PDF oficial) — uso: referencia consultada en su fuente original
- Gartner Hype Cycle (metodología, para uso crítico) — uso: referencia consultada en su fuente original
- McCulloch & Pitts → perceptrón: contexto en Goodfellow et al., *Deep Learning*, cap. 1 — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P57 · Propuesta para el proyecto de investigación de verano de Dartmouth sobre inteligencia artificial	1955	Bautiza el campo y fija su agenda: siete temas que aún organizan buena parte de la investigación.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	citada en las referencias de esta clase · cap. 1
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 1 · obra de referencia de la parte 00

📝 Evaluación completa

❓ Preguntas

1. Define inviernos, resurgimientos y ciclos de expectativas sin usar una marca o framework como definición.
2. Explica la relación entre AI winters, expectativas, evidencia, mercado.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

004 — Agentes racionales, entornos y medidas de desempeño

Parte: 00 — Fundamentos, historia y método científico
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `agent` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **agentes racionales, entornos y medidas de desempeño** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar agentes racionales, entornos y medidas de desempeño usando los conceptos `PEAS`, `racionalidad`, `entorno`, `desempeño`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`PEAS`, `racionalidad`, `entorno`, `desempeño`

🗺️ Ubicación en el mapa de la IA

El marco de agentes racionales (AIMA, cap. 2) es el lenguaje unificador de la IA moderna: permite describir con el mismo vocabulario un termostato, un buscador de rutas, un jugador de ajedrez y un modelo de lenguaje con herramientas. Sustituye la pregunta filosófica "¿piensa?" por la pregunta de ingeniería "¿maximiza su medida de desempeño en su entorno?". Todo el resto del programa — búsqueda, aprendizaje, RL, agentes con LLM — son formas de construir la función del agente.

📖 Fundamentos

🤖 Agente, percepción y función del agente

Un **agente** es cualquier entidad que percibe su entorno mediante **sensores** y actúa sobre él mediante **actuadores**. Formalmente:

```
secuencia de percepciones:  p₁, p₂, ..., pₜ
función del agente:         f: P* → A   (de historiales de percepción a acciones)
programa del agente:        implementación concreta y finita de f
```

La distinción función/programa importa: la función es la especificación matemática (potencialmente una tabla infinita); el programa es el código que la aproxima con memoria y cómputo finitos.

🎯 Racionalidad y medida de desempeño

Un **agente racional** elige, para cada secuencia de percepciones, la acción que **maximiza el valor esperado de la medida de desempeño**, dado el conocimiento previo y las percepciones hasta el momento. Cuatro precisiones críticas:

1. **Racional ≠ omnisciente:** la racionalidad maximiza el resultado *esperado* con la información disponible; no exige conocer el resultado real.
2. **Racional ≠ perfecto:** un agente racional puede obtener malos resultados por mala suerte; se juzga la decisión, no el desenlace.
3. La medida de desempeño debe evaluar **estados del entorno**, no estados del agente ("cuánta suciedad aspiró" es hackeable aspirando y tirando la misma suciedad; "qué tan limpio está el suelo por hora" no).
4. La racionalidad incluye **recopilar información** y **aprender**: ignorar percepciones disponibles es irracional.

📋 Especificación PEAS

Antes de diseñar un agente se especifica su entorno de tareas con **PEAS**:

- **P**erformance (medida de desempeño): qué se maximiza.
- **E**nvironment (entorno): dónde opera.
- **A**ctuators (actuadores): con qué actúa.
- **S**ensors (sensores): qué percibe.

🌎 Dimensiones del entorno

Las propiedades del entorno determinan qué arquitectura de agente es viable:

Dimensión	Extremo fácil	Extremo difícil
Observabilidad	Totalmente observable	Parcialmente observable
N.º de agentes	Un agente	Multiagente (competitivo/cooperativo)
Determinismo	Determinista	Estocástico
Episodicidad	Episódico	Secuencial (las acciones afectan el futuro)
Dinámica	Estático	Dinámico (cambia mientras el agente delibera)
Estados/tiempo	Discreto	Continuo
Conocimiento	Conocido (reglas dadas)	Desconocido (hay que aprenderlas)

El caso más difícil (parcialmente observable, multiagente, estocástico, secuencial, dinámico, continuo, desconocido) es, por ejemplo, conducir en tráfico real.

Taxonomía de programas de agente

1. **Reflejo simple:** reglas condición→acción sobre la percepción actual. Solo funciona con observabilidad total.
2. **Reflejo con estado (basado en modelo):** mantiene un estado interno actualizado con un modelo de transición del mundo; tolera observabilidad parcial.
3. **Basado en metas:** delibera — busca secuencias de acciones que alcanzan una meta explícita (conecta con búsqueda, parte 01).
4. **Basado en utilidad:** compara estados por *cuán deseables* son (función de utilidad), maneja metas en conflicto y riesgo mediante utilidad esperada.
5. **Que aprende:** cualquiera de los anteriores más un elemento de aprendizaje, un crítico y un generador de problemas (exploración). Es la arquitectura general del ML moderno.

Ejemplo trabajado

Especificación PEAS de un agente aspirador en el mundo de dos casillas (A, B) de AIMA:

- **P:** +1 por casilla limpia por paso de tiempo, durante 1000 pasos; −1 por movimiento.
- **E:** dos casillas; la suciedad puede reaparecer con probabilidad 0.1 por paso.
- **A:** `Izquierda`, `Derecha`, `Aspirar`, `NoOp`.
- **S:** posición actual y si la casilla actual está sucia — observabilidad **parcial** (no ve la otra casilla).

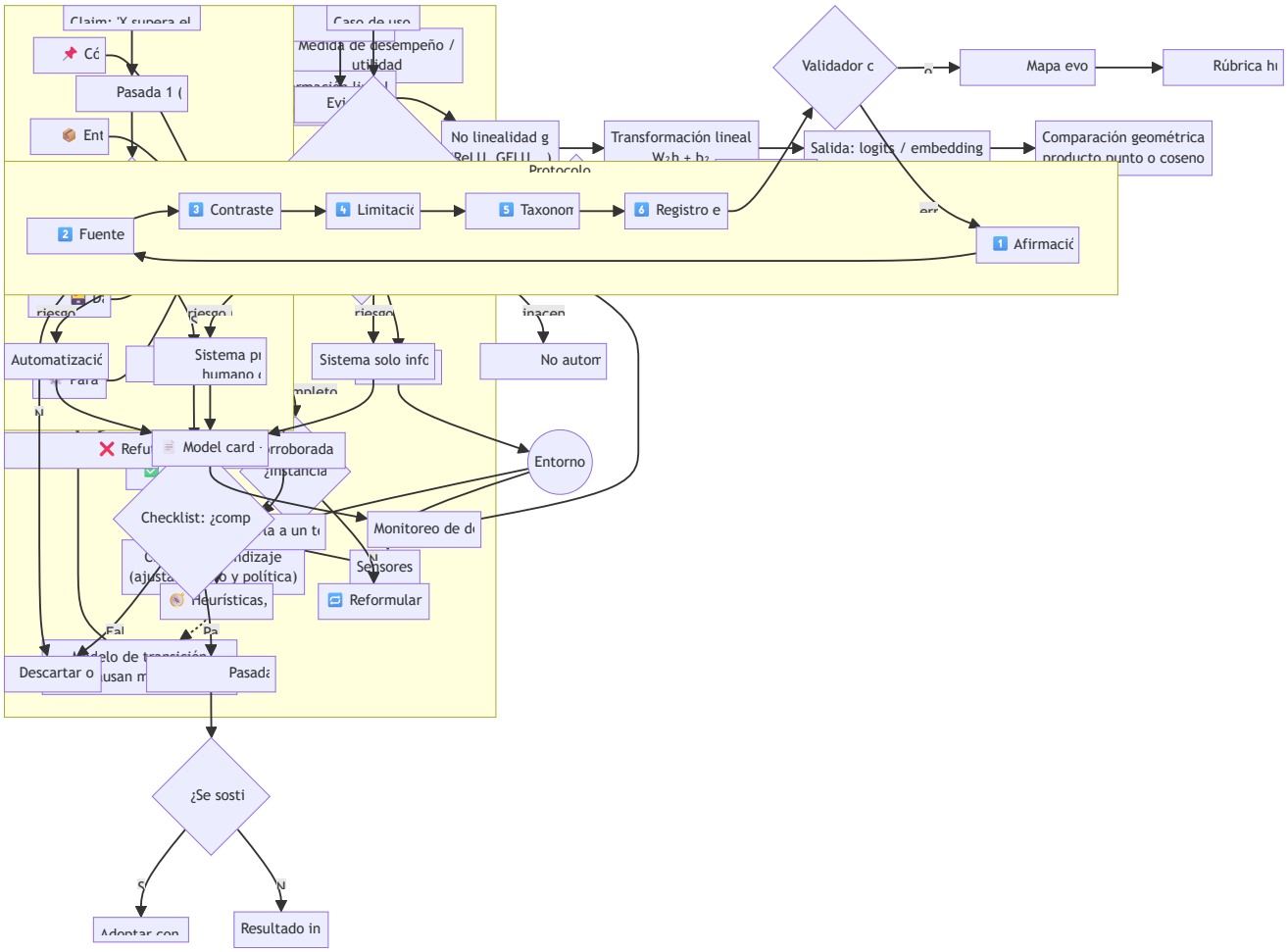
Traza de un agente reflejo con estado, empezando en A, ambas sucias:

t	Percepción	Estado interno (creencia)	Acción	Razón
1	(A, sucia)	B: desconocido	Aspirar	limpiar rinde +1/paso futuro
2	(A, limpia)	B: desconocido	Derecha	recopilar información es racional
3	(B, sucia)	A: limpia (hace 1 paso)	Aspirar	idem t=1
4	(B, limpia)	A: limpia con prob. 0.9	NoOp	moverse cuesta −1 y no hay evidencia de suciedad

Nótese la decisión en t=4: un reflejo simple sin estado oscilaría entre casillas pagando −1 por viaje; el estado interno permite quedarse quieto hasta que la probabilidad de suciedad acumulada justifique el costo del viaje. La racionalidad depende de la medida P: si el movimiento no costara nada, patrullar sería óptimo.

Propiedades y comparación

Arquitectura	Requiere modelo del mundo	Maneja obs. parcial	Maneja metas en conflicto	Costo computacional
Reflejo simple	No	No	No	Mínimo
Reflejo con estado	Sí (transición)	Sí	No	Bajo
Basado en metas	Sí	Sí	No (meta binaria)	Medio (búsqueda)
Basado en utilidad	Sí	Sí	Sí (utilidad esperada)	Alto
Que aprende	Lo aprende	Sí	Sí	Variable + entrenamiento



⚠️ Errores conceptuales frecuentes

1. **"Racional significa que siempre gana."** La racionalidad se define sobre el valor *esperado* con la información disponible; un resultado malo no implica una decisión irracional (ni viceversa).
2. **"La medida de desempeño es un detalle."** Es la especificación completa del objetivo: medidas mal diseñadas producen agentes que optimizan literalmente lo que se midió (aspirar y volcar la suciedad para volver a aspirarla).
3. **"Más deliberación siempre es mejor."** En entornos dinámicos, deliberar tiene costo de oportunidad; un reflejo rápido puede ser más racional que un plan óptimo tardío.

4. **"Los agentes con LLM son otra cosa."** Encajan en el marco: percepciones (contexto, resultados de herramientas), acciones (llamadas a herramientas, texto), medida de desempeño (criterio de éxito de la tarea). El marco expone justamente lo que les falta: medidas de desempeño explícitas y verificables.
5. **"Entorno determinista = agente trivial."** El ajedrez es determinista y totalmente observable, y aun así intratable por fuerza bruta: el tamaño del espacio de estados es una dificultad independiente.

🚀 Del aprendizaje a la operación

Llevar un agente a operación exige: escribir la especificación PEAS como documento revisable antes de codificar; validar la medida de desempeño contra el fenómeno de Goodhart (¿qué comportamiento absurdo la maximizaría?); clasificar el entorno real en las siete dimensiones para dimensionar sensores y frecuencia de decisión; y definir qué hace el agente cuando sus percepciones salen del dominio previsto (fallback a humano, modo seguro). El laboratorio de esta clase ejercita solo el primer eslabón: agente, entorno y medida en versión mínima.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("agent")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Russell, S. & Norvig, P. *Artificial Intelligence: A Modern Approach*, 4.ª ed., cap. 2 — uso: desarrollo extendido del tema
- Turing, A. M. (1950). Computing Machinery and Intelligence (criterio conductual precursor) — uso: fuente primaria del mecanismo estudiado
- Sutton, R. & Barto, A. *Reinforcement Learning: An Introduction*, 2.ª ed., cap. 1 (agente-entorno-recompensa) — uso: desarrollo extendido del tema
- Nilsson, N. (2010). *The Quest for Artificial Intelligence* (PDF oficial) — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P59 · Agentes inteligentes: teoría y práctica	1995	Fija qué es un agente y qué propiedades lo definen, y separa la teoría de las arquitecturas y de los lenguajes que la implementan.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 2 · obra de referencia de la parte 00
Sutton, Richard S. y Barto, Andrew G. — *Reinforcement Learning: An Introduction*	2.ª · 2018	ISBN 9780262039246 · web de la obra	citada en las referencias de esta clase · cap. 1

📝 Evaluación completa

❓ Preguntas

1. Define agentes racionales, entornos y medidas de desempeño sin usar una marca o framework como definición.
2. Explica la relación entre PEAS, racionalidad, entorno, desempeño.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

005 — Vectores, matrices y geometría para IA

Parte: 00 — Fundamentos, historia y método científico
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `optimization` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **vectores, matrices y geometría para ia** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar vectores, matrices y geometría para ia usando los conceptos `vectores`, `matrices`, `producto punto`, `distancia`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`vectores`, `matrices`, `producto punto`, `distancia`

🗺️ Ubicación en el mapa de la IA

El álgebra lineal es el idioma nativo de la IA moderna: datos, parámetros, activaciones y gradientes son vectores y matrices, y casi todo modelo — de la regresión lineal al transformer — es una composición de productos matriciales y no linealidades. Esta clase da la base geométrica que reaparecerá en embeddings y similitud (búsqueda semántica), en descenso de gradiente (parte de optimización) y en la mecánica de atención de los LLM.

📖 Fundamentos

📐 Vectores: tres lecturas del mismo objeto

Un vector $\mathbf{x} \in \mathbb{R}^n$ admite tres interpretaciones intercambiables:

1. **Lista de números:** `x = [x₁, ..., xₙ]` — la vista del programador (array).
2. **Punto/flecha en el espacio:** la vista geométrica — dirección y magnitud.
3. **Objeto representado:** en IA, una fila de datos, una imagen aplanada, un embedding de palabra. La apuesta central del deep learning es que la *semántica* puede codificarse como *geometría*: cosas

parecidas → vectores cercanos.

Operaciones básicas (componente a componente): suma `x + y`, escalado `αx`. Con ellas se definen **combinaciones lineales** $\alpha_1 x_1 + \ldots + \alpha_k x_k$, y de ahí los conceptos de independencia lineal, base y dimensión.

🎯 Producto punto, norma y ángulo

```
producto punto:   x · y = Σᵢ xᵢ yᵢ
norma euclídea:   ‖x‖ = √(x · x)
coseno:           cos θ = (x · y) / (‖x‖ ‖y‖)
distancia:        d(x, y) = ‖x − y‖
```

El producto punto mide *alineación*: positivo si apuntan en direcciones similares, cero si son ortogonales, negativo si se oponen. De él derivan las dos medidas de similitud dominantes en IA: **similitud coseno** (ignora magnitud, estándar en embeddings de texto) y **distancia euclídea** (sensible a magnitud, estándar en clustering). Una neurona artificial computa exactamente `w · x + b`: su "detección" es un producto punto entre la entrada y un patrón aprendido.

🔲 Matrices como transformaciones lineales

Una matriz $A \in \mathbb{R}^{m \times n}$ es una función lineal $\mathbb{R}^n \to \mathbb{R}^m$: `y = Ax`. Leerla como transformación (no como tabla) es el salto conceptual clave:

- Las **columnas de A** son las imágenes de los vectores de la base canónica: dicen a dónde va cada eje.
- La composición de transformaciones es el **producto de matrices**: `B(Ax) = (BA)x`. No conmuta: rotar y luego estirar ≠ estirar y luego rotar.
- Una capa densa de red neuronal es `h = g(Wx + b)`: transformación lineal W, traslación b, no linealidad g. Sin g, apilar capas colapsa a una sola matriz (composición de lineales es lineal) — por eso las activaciones no lineales son imprescindibles.

Dimensiones: `(m×n)·(n×k) → (m×k)`; el índice interior debe coincidir. La mayoría de bugs de shape en NumPy/PyTorch son violaciones de esta regla.

🧭 Nociones geométricas que reaparecen en IA

- **Proyección** de x sobre u (unitario): `proj_u(x) = (x·u)u` — base de PCA y de "cuánto de este concepto hay en este embedding".
- **Hiperplano** `w·x + b = 0`: frontera de decisión de un clasificador lineal; w es su normal. Separabilidad lineal = existe un hiperplano que separa las clases (la limitación XOR del perceptrón es exactamente esto).
- **Maldición de la dimensionalidad:** en dimensión alta, las distancias euclídeas se concentran y los volúmenes se vacían; la similitud coseno y las estructuras de índice aproximado (ANN) existen en parte por esto.

🧮 Ejemplo trabajado

Similitud entre tres "documentos" representados como vectores de conteo sobre el vocabulario `[ia, datos, fútbol]`:

```
d₁ = [2, 3, 0]   (habla de IA y datos)
d₂ = [1, 2, 0]   (habla de IA y datos, más corto)
d₃ = [0, 1, 4]   (habla sobre todo de fútbol)
```

Paso a paso para (d_1, d_2):

```
d₁ · d₂ = 2·1 + 3·2 + 0·0 = 8
‖d₁‖ = √(4+9+0) = √13 ≈ 3.606
‖d₂‖ = √(1+4+0) = √5  ≈ 2.236
cos(d₁,d₂) = 8 / (3.606·2.236) ≈ 8/8.062 ≈ 0.992   → casi idénticos en tema
```

Para (d_1, d_3): `d₁·d₃ = 0+3+0 = 3`, `‖d₃‖ = √17 ≈ 4.123`, `cos ≈ 3/(3.606·4.123) ≈ 0.202` → temas distintos.

Obsérvese que la distancia euclídea `‖d₁−d₂‖ = ‖[1,1,0]‖ = √2 ≈ 1.41` es *mayor* que cero aunque los documentos traten exactamente lo mismo: el coseno corrige la diferencia de longitud. Este es el motivo por el que los motores de embeddings usan coseno por defecto.

📊 Propiedades y comparación

Medida	Fórmula	Sensible a magnitud	Rango	Uso típico en IA
Producto punto	$\Sigma x_i y_i$	Sí	$(-\infty, \infty)$	Capas densas, atención (QK^T)
Similitud coseno	$x \cdot y / (\|x\|\|y\|)$	No	$[-1, 1]$	Embeddings, búsqueda semántica
Distancia euclídea	$\|x-y\|$	Sí	$[0, \infty)$	k-means, k-NN en dimensión baja
Distancia Manhattan	$\Sigma\|x_i-y_i\|$	Sí	$[0, \infty)$	Datos dispersos, robustez a outliers

⚠️ Errores conceptuales frecuentes

1. **"El producto de matrices es componente a componente."** Ese es el producto de Hadamard; el producto matricial estándar es composición de transformaciones (filas por columnas) y no conmuta.
2. **"Coseno y euclídea dan el mismo ranking."** Solo si todos los vectores tienen la misma norma; con normas distintas los rankings divergen (ver ejemplo trabajado).

3. **"Más capas lineales = más capacidad."** Sin no linealidad entre ellas, cualquier pila de capas lineales equivale a una sola transformación lineal.
4. **"La intuición de 2D/3D escala a dimensión 768."** En dimensión alta casi todos los vectores aleatorios son casi ortogonales y las distancias se concentran; hay que razonar con álgebra, no con dibujos.
5. **"Un embedding cercano implica significado idéntico."** Cercanía geométrica refleja coocurrencia estadística en los datos de entrenamiento; puede codificar sesgos o asociaciones espurias, no sinonimia garantizada.

🚀 Del aprendizaje a la operación

En sistemas reales, esta base se convierte en: elegir métrica de similitud y normalización coherentes en *todo* el pipeline (indexar con coseno y consultar con euclídea es un bug clásico de RAG); vigilar shapes y convenciones fila/columna entre bibliotecas; usar operaciones vectorizadas (BLAS/GPU) en lugar de bucles Python — una diferencia de órdenes de magnitud; y validar que la geometría del embedding realmente separa las clases del dominio propio antes de construir encima.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("optimization")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Deisenroth, Faisal & Ong. *Mathematics for Machine Learning*, caps. 2-3 (PDF oficial gratuito) — uso: referencia consultada en su fuente original
- Goodfellow, Bengio & Courville. *Deep Learning*, cap. 2: Linear Algebra — uso: desarrollo extendido del tema
- 3Blue1Brown. *Essence of Linear Algebra* (serie visual) — uso: referencia consultada en su fuente original
- Strang, G. MIT OCW 18.06 Linear Algebra — uso: referencia consultada en su fuente original
- NumPy: documentación oficial de álgebra lineal — uso: referencia consultada en su fuente original

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P53 · Sobre las líneas y planos de ajuste más próximo a sistemas de puntos en el espacio	1901	La primera respuesta al problema de resumir una nube de puntos con menos dimensiones sin privilegiar ninguna variable.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	citada en las referencias de esta clase · cap. 2
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 00 · capítulos de introducción y de agentes racionales

📝 Evaluación completa

❓ Preguntas

1. Define vectores, matrices y geometría para ia sin usar una marca o framework como definición.
2. Explica la relación entre vectores, matrices, producto punto, distancia.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

006 — Probabilidad, incertidumbre y estadística básica

Parte: 00 — Fundamentos, historia y método científico
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `probability` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **probabilidad, incertidumbre y estadística básica** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar probabilidad, incertidumbre y estadística básica usando los conceptos `probabilidad`, `Bayes`, `distribuciones`, `incertidumbre`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`probabilidad`, `Bayes`, `distribuciones`, `incertidumbre`

🗺️ Ubicación en el mapa de la IA

La probabilidad es el cálculo de la incertidumbre, y la IA opera casi siempre bajo incertidumbre: sensores ruidosos, datos incompletos, mundos parcialmente observables. El giro probabilístico de los años 90 (redes bayesianas, métodos estadísticos) sacó al campo de su segundo invierno, y hoy todo clasificador emite probabilidades y todo LLM es, literalmente, una distribución de probabilidad sobre el siguiente token. Esta clase da el mínimo necesario para las partes de machine learning y para leer métricas con honestidad.

📖 Fundamentos

🎲 Axiomas y vocabulario

Una probabilidad asigna a cada evento A un número P(A) que cumple (Kolmogórov):

```
1. P(A) ≥ 0
2. P(Ω) = 1              (algo del espacio muestral ocurre)
3. P(A ∪ B) = P(A) + P(B)   si A y B son disjuntos
```

De ahí: P(¬A) = 1 − P(A), y la **regla de la suma general** P(A∪B) = P(A) + P(B) − P(A∩B). Dos lecturas filosóficas conviven: frecuentista (límite de frecuencias relativas) y bayesiana (grado de creencia coherente); en la práctica de IA se usan ambas según convenga al problema.

🔗 Probabilidad condicional e independencia

```
P(A|B) = P(A∩B) / P(B)          (probabilidad de A sabiendo que ocurrió B)
regla del producto:  P(A∩B) = P(A|B) P(B)
independencia:       P(A∩B) = P(A) P(B)   ⇔   P(A|B) = P(A)
```

Condicionar es *actualizar el universo*: al saber B, el espacio muestral se reduce a B. La independencia es un supuesto de modelado, no un hecho por defecto: asumirla donde no existe (p. ej., entre features correlacionadas) es una fuente clásica de modelos sobreconfiados.

🔁 Teorema de Bayes

```
P(H|E) = P(E|H) · P(H) / P(E)

posterior = verosimilitud × prior / evidencia
```

Bayes invierte la dirección del condicional: de "probabilidad de la evidencia dada la hipótesis" (que el modelo o el test médico conocen) a "probabilidad de la hipótesis dada la evidencia" (que es lo que uno quiere). P(E) se expande por probabilidad total: `P(E) = P(E|H)P(H) + P(E|¬H)P(¬H)`. Es el fundamento de los clasificadores naive Bayes, del filtrado de spam, de la inferencia en redes bayesianas y del razonamiento diagnóstico.

📊 Variables aleatorias, esperanza y varianza

Una variable aleatoria X asigna números a resultados. Sus resúmenes centrales:

```
esperanza:  E[X] = Σ x · P(X=x)          (media ponderada por probabilidad)
varianza:   Var(X) = E[(X − E[X])²]      (dispersión alrededor de la media)
desviación estándar: σ = √Var(X)
```

Distribuciones que hay que reconocer: **Bernoulli** (un ensayo sí/no), **binomial** (número de éxitos en n ensayos), **uniforme**, **normal/gaussiana** (suma de muchos efectos pequeños independientes — teorema central del límite). La **ley de los grandes números** garantiza que el promedio muestral converge a E[X]: es la licencia matemática para estimar probabilidades simulando (método Monte Carlo), que es exactamente lo que hace el laboratorio de esta clase con una semilla fija.

⚠️ Estadística mínima para leer resultados

- Un **estimador** calculado sobre una muestra tiene **error muestral**: reportar una métrica sin tamaño de muestra ni intervalo es reportar ruido potencial.
- **Correlación no es causalidad:** dos variables pueden covariar por una causa común o por azar (con suficientes comparaciones, el azar *garantiza* correlaciones espurias).

- La media es sensible a outliers; la mediana no. Elegir el resumen según la distribución.

🧮 Ejemplo trabajado

El clásico problema del test diagnóstico, que casi todo el mundo responde mal la primera vez. Una enfermedad afecta al 1 % de la población. El test detecta al 90 % de los enfermos (sensibilidad) y da falso positivo en el 9 % de los sanos. Si una persona da positivo, ¿cuál es la probabilidad de que esté enferma?

Con 10 000 personas, en números enteros:

```
Enfermos:  100   → positivos verdaderos: 100 × 0.90 =  90
Sanos:    9900   → falsos positivos:    9900 × 0.09 = 891

P(enfermo | positivo) = 90 / (90 + 891) = 90/981 ≈ 0.092  →  ~9 %
```

Por Bayes directamente: P(H|E) = (0.90 · 0.01) / (0.90·0.01 + 0.09·0.99) = 0.009/0.0981 ≈ **0.092**. La intuición dice "90 %"; la respuesta correcta es ~9 %, porque el **prior** (1 %) domina: hay muchos más sanos que pueden dar falso positivo que enfermos que den verdadero positivo. Moraleja directa para IA: un clasificador "90 % preciso" sobre una clase rara produce mayoritariamente falsas alarmas — por eso exigimos precisión/recall y no solo accuracy.

📊 Propiedades y comparación

Concepto	Qué responde	Trampa típica
P(A)	Frecuencia/creencia marginal	Ignorar que depende de la población de referencia
P(A|B)	Creencia actualizada por evidencia	Confundir P(A|B) con P(B|A) (falacia del fiscal)
Bayes	Invierte el condicional con el prior	Omitir el prior (ignorar la tasa base)
E[X]	Valor promedio a largo plazo	Usarla con distribuciones sin media estable o con outliers
Var(X)	Dispersión esperada	Reportar medias sin dispersión ni n
Monte Carlo	Estima E[X] simulando	Olvidar la semilla → resultados irreproducibles

⚠️ Errores conceptuales frecuentes

1. **Confundir P(A|B) con P(B|A).** "El 90 % de los enfermos da positivo" no implica "el 90 % de los positivos está enfermo" — la diferencia la pone el prior (ver ejemplo).

2. **Ignorar la tasa base.** Evaluar un detector de fraude, spam o enfermedad rara por su accuracy global: un modelo que dice siempre "no" acierta 99 % y es inútil.
3. **"Independiente" como valor por defecto.** Multiplicar probabilidades solo es válido bajo independencia; features de un mismo individuo raramente lo son.
4. **Tratar la probabilidad del modelo como calibrada.** Que un softmax diga 0.97 no significa que el modelo acierte el 97 % de las veces que dice 0.97; la calibración se mide, no se asume.
5. **Confundir significancia con importancia.** Con n enorme, diferencias triviales se vuelven "significativas"; con n pequeño, efectos reales quedan invisibles. Siempre reportar tamaño de efecto y n.

🚀 Del aprendizaje a la operación

En producción este material se convierte en: medir la **calibración** de las probabilidades del modelo (reliability diagrams, ECE) antes de usarlas para decidir; fijar umbrales de decisión según costos asimétricos de falso positivo/negativo, no en 0.5 por defecto; monitorear el cambio del prior en el tiempo (la tasa base de fraude de ayer no es la de mañana); y acompañar toda métrica reportada de su n, su intervalo y la semilla del experimento que la produjo.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("probability")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Russell, S. & Norvig, P. *AIMA*, 4.ª ed., caps. 12-13 (quantifying uncertainty) — uso: desarrollo extendido del tema
- Deisenroth, Faisal & Ong. *Mathematics for Machine Learning*, cap. 6 (PDF oficial gratuito) — uso: referencia consultada en su fuente original
- Goodfellow, Bengio & Courville. *Deep Learning*, cap. 3: Probability and Information Theory — uso: desarrollo extendido del tema
- Seeing Theory — visualizaciones interactivas de probabilidad (Brown University) — uso: referencia consultada en su fuente original
- Ioannidis, J. (2005). Why Most Published Research Findings Are False. *PLoS Medicine* — uso: fuente primaria del mecanismo estudiado

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P55 · Una teoría matemática de la comunicación	1948	Define la información como reducción de incertidumbre y le pone unidad, cota y límite: el bit, la entropía y la capacidad del canal.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Goodfellow, Ian, Bengio, Yoshua y Courville, Aaron — *Deep Learning*	2016	ISBN 9780262035613 · web de la obra	citada en las referencias de esta clase · cap. 3
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · caps. 12-13 · obra de referencia de la parte 00

📝 Evaluación completa

❓ Preguntas

1. Define probabilidad, incertidumbre y estadística básica sin usar una marca o framework como definición.
2. Explica la relación entre probabilidad, Bayes, distribuciones, incertidumbre.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

007 — Lógica, algoritmos y complejidad computacional

Parte: 00 — Fundamentos, historia y método científico
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `logic` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **lógica, algoritmos y complejidad computacional** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar lógica, algoritmos y complejidad computacional usando los conceptos `lógica`, `algoritmos`, `complejidad`, `decidibilidad`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`lógica`, `algoritmos`, `complejidad`, `decidibilidad`

🗺️ Ubicación en el mapa de la IA

La lógica fue el primer lenguaje de la IA (el Logic Theorist de 1956 demostraba teoremas), y la teoría de la complejidad explica por qué esa primera IA chocó contra un muro: muchos problemas de razonamiento son intratables en el peor caso — la "explosión combinatoria" que diagnosticó el informe Lighthill. Esta clase da los tres instrumentos para razonar sobre límites: lógica formal (qué se puede expresar), análisis de algoritmos (cuánto cuesta) y computabilidad (qué es imposible), fundamentos de la parte de búsqueda y de todo el programa.

📖 Fundamentos

🔣 Lógica proposicional: sintaxis y semántica

La lógica proposicional maneja proposiciones atómicas (p, q) y conectivos (¬, ∧, ∨, →, ↔). Conceptos clave:

- **Modelo:** una asignación de verdad a cada átomo. Con n átomos hay 2^n modelos.

- **Consecuencia lógica (KB ⊨ α):** α es verdadera en *todos* los modelos donde KB es verdadera. Verificarlo por tabla de verdad cuesta $O(2^n)$ — primera aparición del muro exponencial.
- **Inferencia:** procedimientos como *modus ponens* (de `p` y `p→q`, concluir `q`) y resolución. Un sistema de inferencia útil debe ser **correcto** (solo deriva consecuencias verdaderas) y ojalá **completo** (deriva todas).
- **SAT:** decidir si una fórmula tiene algún modelo que la haga verdadera. Es el problema NP-completo canónico (Cook, 1971) y, a la vez, la base de solvers industriales que resuelven instancias enormes en la práctica.

La lógica de primer orden añade objetos, relaciones y cuantificadores (∀, ∃); gana expresividad y pierde decidibilidad: la consecuencia lógica es solo semidecidible.

⏱️ Algoritmos y notación asintótica

Un algoritmo es un procedimiento finito, definido y efectivo. Su costo se describe con notación asintótica sobre el tamaño de entrada n:

```
O(g)  : cota superior     ("no crece más rápido que g")
Ω(g)  : cota inferior
Θ(g)  : cota ajustada     (superior e inferior a la vez)
```

Jerarquía de crecimiento que hay que tener interiorizada:

```
O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(n³) < O(2ⁿ) < O(n!)
```

La frontera práctica está entre polinomial y exponencial: duplicar la velocidad del hardware suma una constante al tamaño tratable de un problema exponencial (2^n), mientras que multiplica el de uno lineal. Ningún hardware rescata a un algoritmo exponencial.

🧩 P, NP y NP-completitud

- **P:** problemas de decisión resolubles en tiempo polinomial.
- **NP:** problemas cuya solución propuesta es *verificable* en tiempo polinomial.
- **NP-completo:** los más difíciles de NP; todo problema de NP se reduce a ellos en tiempo polinomial. SAT fue el primero (teorema de Cook-Levin, 1971); planificación, muchos problemas de scheduling y variantes de inferencia en IA son NP-duros.
- Si P = NP se desconoce (problema del milenio); la evidencia práctica sugiere P ≠ NP.

Consecuencia para IA: la intratabilidad del peor caso no prohíbe resolver *instancias reales* — heurísticas, aproximaciones y solvers SAT/SMT modernos lo hacen a diario — pero obliga a renunciar a garantías universales de optimalidad o tiempo.

🛑 Computabilidad: lo imposible, no solo lo caro

Por encima de "caro" está "imposible": Turing (1936) demostró que el **problema de la parada** (¿este programa termina con esta entrada?) es **indecidible** — no existe algoritmo que lo responda para todos los casos. Del teorema de Rice se sigue que toda propiedad semántica no trivial de programas es indecidible. Esto acota lo que cualquier IA puede hacer: verificar comportamiento arbitrario de software, garantizar ausencia

total de errores o predecir su propia conducta en general son problemas sin solución algorítmica exacta y universal.

🧮 Ejemplo trabajado

Parte A — inferencia proposicional. KB = {p→q, q→r, p}. ¿KB ⊨ r?

```
1. p          (hecho)
2. p → q      (regla)      ⇒ modus ponens: q
3. q → r      (regla)      ⇒ modus ponens: r        ■  KB ⊨ r
```

Verificación semántica: en toda asignación donde p=V, p→q=V y q→r=V, forzosamente q=V y r=V. Con 3 átomos habría $2^3=8$ modelos que revisar por tabla; el encadenamiento llegó en 2 pasos — la inferencia sintáctica evita enumerar modelos.

Parte B — el muro exponencial en números. Un solver ingenuo de SAT prueba las 2^n asignaciones, a 10^9 asignaciones/segundo:

n átomos	2^n	Tiempo
20	~10^6	1 ms
40	~10^{12}	18 minutos
60	~10^{18}	36 años
80	~10^{24}	38 millones de años

Sumar 20 variables multiplica el tiempo por un millón. Esta tabla *es* la explosión combinatoria de Lighthill (clase 003) y la razón de ser de las heurísticas (parte 01).

📊 Propiedades y comparación

Formalismo / clase	Expresividad	Decidible	Costo típico	Uso en IA
Lógica proposicional	Baja (hechos atómicos)	Sí	SAT: NP-completo	Solvers, verificación, planificación
Lógica de primer orden	Alta (objetos, relaciones)	Semidecidible	Sin cota general	Representación de conocimiento
P (polinomial)	—	Sí	n, n log n, n^2	Grafos, ordenación, inferencia acotada
NP-completo	—	Sí (caro)	Exponencial en el peor caso	SAT, planificación, scheduling
Indecidible	—	No	∞	Límite absoluto (parada, Rice)

⚠️ Errores conceptuales frecuentes

1. **"NP significa 'no polinomial'."** NP = *verificable* en tiempo polinomial (nondeterministic polynomial). P ⊆ NP, y si P=NP todo NP sería polinomial.
2. **"NP-completo = imposible en la práctica."** SAT solvers modernos resuelven instancias industriales con millones de variables; la dureza es del *peor caso*, no de toda instancia.
3. **"Con más hardware/nube desaparece el problema exponencial."** Duplicar cómputo suma ~1 al n tratable de 2^n; el crecimiento exponencial devora cualquier constante.
4. **"La indecidibilidad es lo mismo que la intratabilidad."** Intratable = caro pero computable; indecidible = no existe algoritmo, con cualquier recurso, para todos los casos.
5. **"O(g) describe el caso típico."** Es una cota superior asintótica del peor caso salvo que se diga otra cosa; quicksort es $O(n^2)$ en el peor caso y $\Theta(n \log n)$ en promedio.

🚀 Del aprendizaje a la operación

Profesionalmente, esto se traduce en: clasificar el problema antes de codificar (¿es polinomial, NP-duro, indecidible?) porque el veredicto decide entre solución exacta, heurística o rediseño; presupuestar el crecimiento de la entrada (lo que funciona con $n=10^3$ puede morir con $n=10^6$); usar solvers maduros (SAT/SMT/MILP) en lugar de búsqueda artesanal; y desconfiar por principio de cualquier herramienta que prometa verificar propiedades semánticas arbitrarias de programas — el teorema de Rice ya dictó sentencia.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("logic")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Turing, A. M. (1936). On Computable Numbers, with an Application to the Entscheidungsproblem — uso: fuente primaria del mecanismo estudiado

- Cook, S. (1971). The Complexity of Theorem-Proving Procedures (SAT es NP-completo) — uso: fuente primaria del mecanismo estudiado
- Russell, S. & Norvig, P. *AIMA*, 4.ª ed., caps. 7-8 (agentes lógicos) y apéndice A (complejidad) — uso: desarrollo extendido del tema
- Clay Mathematics Institute. P vs NP (problema del milenio) — uso: referencia consultada en su fuente original
- Nilsson, N. (2010). *The Quest for Artificial Intelligence*, caps. sobre razonamiento lógico (PDF oficial) — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P54 · Un cálculo lógico de las ideas inmanentes en la actividad nerviosa	1943	Establece que una red de neuronas de umbral puede calcular cualquier función lógica: el puente entre biología y computación.	notebook
P58 · La informática como indagación empírica: símbolos y búsqueda	1976	Enuncia las dos hipótesis que resumen veinte años de IA simbólica: el sistema de símbolos físicos y la búsqueda heurística.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · caps. 7-8 · obra de referencia de la parte 00

📝 Evaluación completa

❓ Preguntas

1. Define lógica, algoritmos y complejidad computacional sin usar una marca o framework como definición.
2. Explica la relación entre lógica, algoritmos, complejidad, decidibilidad.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

008 — Datos, evidencia, hipótesis y falsabilidad

Parte: 00 — Fundamentos, historia y método científico
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `evaluation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **datos, evidencia, hipótesis y falsabilidad** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar datos, evidencia, hipótesis y falsabilidad usando los conceptos `datos`, `hipótesis`, `falsabilidad`, `reproducibilidad`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`datos`, `hipótesis`, `falsabilidad`, `reproducibilidad`

🗺️ Ubicación en el mapa de la IA

La IA es una disciplina empírica: sus afirmaciones ("este modelo es mejor", "esta técnica generaliza") se sostienen o caen con experimentos. Esta clase importa el criterio de demarcación de Popper — falsabilidad — al trabajo con modelos y datos, y es el fundamento metodológico de los entornos reproducibles (clase 009), de la lectura crítica de papers (clase 010) y de cada laboratorio del programa, cuyo contrato JSON separa explícitamente `evidence` de `limitations`.

📖 Fundamentos

🔍 Falsabilidad como criterio de demarcación

Para Popper, una hipótesis es **científica** si y solo si existe alguna observación posible que la refutaría. "Este modelo alcanza ≥ 0.85 de F1 en datos de 2025 no vistos" es falsable; "este modelo entiende el lenguaje" no lo es hasta que se traduzca a predicciones observables. Dos consecuencias operativas:

- La ciencia avanza por **conjeturas y refutaciones**: proponer hipótesis audaces y someterlas a los tests más duros disponibles, no buscar confirmaciones.

- Ninguna cantidad de confirmaciones *demuestra* una hipótesis universal (problema de la inducción de Hume); una sola refutación sólida la derriba. La asimetría es el motor.

🧪 De la pregunta al experimento

Estructura mínima de una hipótesis experimental en IA:

```
H: "El sistema/técnica X supera al baseline B en la métrica M
    sobre la población de datos D, por al menos δ."

Falsable porque: si M(X) − M(B) < δ en una muestra representativa de D,  H queda refutada.
```

Cada componente es obligatorio: sin baseline B no hay comparación; sin población D definida no se sabe a qué generaliza; sin margen δ cualquier diferencia de ruido "confirma"; sin métrica M fijada *antes* de mirar los resultados, se cae en la selección post-hoc.

📊 Jerarquía de la evidencia en IA

No toda evidencia pesa igual. De más débil a más fuerte:

1. **Anécdota / demo:** un ejemplo elegido a mano. Sirve para ilustrar, no para concluir.
2. **Evaluación en muestra de entrenamiento:** inválida — memorizar no es generalizar.
3. **Held-out interno:** partición de test del mismo dataset. Mínimo aceptable; vulnerable a fuga de información (*leakage*) y a sobreajuste al benchmark.
4. **Datos externos / fuera de distribución:** otra fuente, otro periodo temporal.
5. **Replicación independiente:** otro equipo, otro código, mismos resultados. El patrón oro; su escasez en ML es la "crisis de reproducibilidad" documentada por Kapoor y Narayanan (2023), que hallaron leakage en cientos de papers aplicados.

🕳️ Patologías del razonamiento con datos

- **Fuga de datos (leakage):** información del test contamina el entrenamiento (duplicados, features derivadas del futuro, normalizar antes de particionar). Produce métricas ficticiamente altas que colapsan en producción.
- **p-hacking / garden of forking paths:** probar configuraciones hasta que una "funciona" y reportar solo esa. Con 20 variantes y α=0.05, se espera un falso positivo por puro azar.
- **HARKing** (*Hypothesizing After Results are Known*): presentar un hallazgo exploratorio como si fuera la hipótesis original. Legítimo como exploración, fraudulento como confirmación.
- **Sesgo de supervivencia y de publicación:** solo se publican los experimentos que salieron bien; el registro público sobreestima sistemáticamente el efecto real (Ioannidis, 2005).
- **Goodhart:** cuando la métrica se vuelve objetivo, deja de medir lo que medía — sobreajuste comunitario a benchmarks públicos incluido.

✅ Contrato de evidencia del programa

Cada laboratorio de este repositorio emite un JSON con `seed`, `evidence` y `limitations`. Es una implementación en miniatura del criterio popperiano: la evidencia es inspeccionable y reproducible (misma semilla → mismo resultado), y las limitaciones declaran de antemano dónde la conclusión deja de valer — el equivalente a especificar qué observación refutaría el claim.

🧮 Ejemplo trabajado

Un equipo afirma: "nuestro clasificador de fraude tiene accuracy 0.99". Auditemos con números. El dataset tiene 100 000 transacciones, 1 000 fraudulentas (1 %).

```
Modelo trivial "nunca es fraude":  acierta 99 000 / 100 000 = 0.99 de accuracy
                                   recall sobre fraude = 0/1000 = 0.0
```

El claim es literalmente cierto y prácticamente vacío: no supera al baseline trivial. Reformulación falsable correcta:

```
H: "El modelo alcanza recall ≥ 0.70 con precisión ≥ 0.80 sobre transacciones
    de un mes posterior al periodo de entrenamiento (validación temporal)."
```

Supongamos que en ese mes hay 900 fraudes y el modelo marca 950 transacciones, de las cuales 720 son fraude real: precisión = 720/950 ≈ 0.758 → **H queda refutada** (0.758 < 0.80) aunque recall = 720/900 = 0.80 cumpla. La refutación es informativa: dice exactamente qué mejorar y evita desplegar un sistema que generaría ~230 falsas alarmas mensuales.

📊 Propiedades y comparación

Nivel de evidencia	Ejemplo	¿Falsable?	Riesgo principal
Demo elegida a mano	"Mira cómo responde bien"	No	Cherry-picking
Métrica en entrenamiento	acc=0.99 (train)	Sí, pero irrelevante	Memorización
Held-out interno	acc en test split	Sí	Leakage, sobreajuste al benchmark
Validación temporal/externa	Datos de otro periodo	Sí	Deriva de distribución (informativa)
Replicación independiente	Otro equipo reproduce	Sí	Costo; escasez de incentivos

⚠️ Errores conceptuales frecuentes

1. **"Los datos hablan por sí mismos."** Los datos solo responden preguntas bien planteadas; sin hipótesis previa, cualquier patrón encontrado puede ser ruido con buena prensa.
2. **"Mi hipótesis fue confirmada, luego es verdadera."** La corroboración no es prueba; solo indica que la hipótesis sobrevivió a *ese* test. La confianza se gana con tests progresivamente más duros.

3. **"Más accuracy = mejor modelo."** Sin baseline, tasa base, métrica apropiada al costo de error y población definida, la cifra no significa nada (ver ejemplo trabajado).
4. **"Explorar datos está mal."** La exploración es legítima y necesaria; el error es disfrazar hallazgos exploratorios de confirmaciones (HARKing). La solución es declarar qué fue exploratorio y validar en datos frescos.
5. **"La reproducibilidad es burocracia."** Es la condición mínima para que un resultado sea evidencia y no anécdota: sin semilla, versión y datos fijados, ni siquiera el propio autor puede verificar su claim.

🚀 Del aprendizaje a la operación

En un flujo profesional esto se institucionaliza: hipótesis y métrica de éxito escritas *antes* del experimento (pre-registro interno); particiones temporales y auditoría de leakage como paso obligatorio del pipeline; un registro de experimentos fallidos (los silencios sesgan tanto como los éxitos); y monitoreo post-despliegue que trata cada predicción en producción como un test continuo de la hipótesis del modelo — con criterios de rollback definidos de antemano.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("evaluation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Popper, K. — entrada en la Stanford Encyclopedia of Philosophy (falsabilidad y demarcación) — uso: referencia consultada en su fuente original
- Ioannidis, J. (2005). Why Most Published Research Findings Are False. *PLoS Medicine* — uso: fuente primaria del mecanismo estudiado
- Kapoor, S. & Narayanan, A. (2023). Leakage and the Reproducibility Crisis in ML-based Science — uso: fuente primaria del mecanismo estudiado
- Sandve et al. (2013). Ten Simple Rules for Reproducible Computational Research. *PLoS Comp Bio* — uso: fuente primaria del mecanismo estudiado
- Pineau et al. (2021). Improving Reproducibility in Machine Learning Research — uso: fuente primaria del mecanismo estudiado

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P60 · Por qué la mayoría de los hallazgos publicados son falsos	2005	Muestra con un modelo explícito que la probabilidad de que un hallazgo publicado sea cierto depende del diseño y de los incentivos, no del valor p.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 00 · capítulos de introducción y de agentes racionales

📝 Evaluación completa

❓ Preguntas

1. Define datos, evidencia, hipótesis y falsabilidad sin usar una marca o framework como definición.
2. Explica la relación entre datos, hipótesis, falsabilidad, reproducibilidad.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;

- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

009 — Entornos Python, Git y experimentos reproducibles

Parte: 00 — Fundamentos, historia y método científico
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `observability` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **entornos python, git y experimentos reproducibles** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar entornos python, git y experimentos reproducibles usando los conceptos `Python`, `Git`, `semillas`, `entornos`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`Python`, `Git`, `semillas`, `entornos`

🗺️ Ubicación en el mapa de la IA

Si la clase 008 estableció que un claim sin experimento reproducible es una anécdota, esta clase da la infraestructura para que los experimentos *sean* reproducibles: entornos de Python aislados y declarados, control de versiones con Git, y gestión explícita de la aleatoriedad con semillas. Es la caja de herramientas silenciosa detrás de cada laboratorio del programa (todos aceptan `seed` y devuelven un JSON verificable) y de cualquier práctica seria de ML, donde "funciona en mi máquina" es el modo de fallo más caro.

📖 Fundamentos

🚦 Los cuatro niveles de reproducibilidad

```
Nivel 0 — Repetibilidad:    yo, en mi máquina, hoy → mismo resultado.
Nivel 1 — Reproducibilidad: otra persona, con mi código y datos → mismo resultado.
Nivel 2 — Replicabilidad:   otro equipo, con SU implementación → misma conclusión.
Nivel 3 — Generalización:   la conclusión sobrevive con otros datos/dominios.
```

Esta clase asegura los niveles 0-1. Requieren fijar cinco cosas: **código** (Git), **dependencias** (entorno declarado), **datos** (versión e integridad), **aleatoriedad** (semillas) y **configuración** (parámetros registrados, no hardcodeados a medias).

🐍 Entornos virtuales de Python

Un entorno virtual (`venv`) es un directorio con un intérprete y un `site-packages` propios: aísla las dependencias de un proyecto de las del sistema y de otros proyectos.

```
python -m venv .venv                    # crear
.venv\Scripts\activate                  # activar (Windows)
source .venv/bin/activate               # activar (Unix)
pip install -e .                        # instalar el proyecto en modo editable
pip freeze > requirements-lock.txt      # congelar versiones exactas
```

Distinción crítica: `requirements.txt` / `pyproject.toml` declaran dependencias *directas* con rangos ("numpy>=1.26"); un **lockfile** (`pip freeze`, `uv lock`, `poetry.lock`) congela el grafo *completo* con versiones exactas. La reproducibilidad exige el lockfile: dos instalaciones con el mismo rango en fechas distintas pueden resolver versiones diferentes y cambiar resultados numéricos.

🌱 Aleatoriedad y semillas

Los generadores pseudoaleatorios (PRNG) son deterministas: una **semilla** fija toda la secuencia. `random.seed(42)` y `numpy.random.default_rng(42)` hacen el experimento repetible. Advertencias honestas:

- Cada biblioteca tiene su propio PRNG: fijar `random` no fija NumPy ni PyTorch; hay que sembrar cada fuente que se use.
- La semilla garantiza repetibilidad, **no validez**: un resultado que solo se sostiene con la semilla 42 es ruido. La práctica correcta es reportar media y dispersión sobre varias semillas.
- En GPU, algunas operaciones son no deterministas por diseño (atomics, reducciones paralelas); el determinismo total puede exigir flags específicos y costar rendimiento.

🌳 Git: instantáneas verificables del código

Git guarda **commits**: instantáneas inmutables del árbol de archivos, identificadas por un hash SHA que depende del contenido y de la historia. Para experimentos:

```
git init / clone          # crear u obtener el repositorio
git add -p                # revisar QUÉ se incluye, fragmento a fragmento
git commit -m "..."       # instantánea con mensaje que explica el porqué
git tag exp-2026-07-29    # marcar el estado exacto de un experimento
git diff / log / show     # auditar qué cambió entre dos resultados
```

El hash del commit es el eslabón que une un número en un informe con el código exacto que lo produjo: un resultado sin commit asociado no es auditable. Las ramas permiten aislar experimentos; `.gitignore` mantiene fuera datos pesados, secretos y artefactos derivados (los datos se versionan con herramientas dedicadas o con checksums registrados).

📄 El contrato experimental mínimo

Todo experimento del programa registra, como mínimo:

```
{commit, entorno (lockfile), semilla(s), parámetros, datos+versión, métrica, fecha}
```

El laboratorio de esta clase (`run_lab("observability", seed=...)`) implementa la versión mínima: mismo seed → mismo JSON, y el resultado incluye la evidencia y las limitaciones declaradas. Ese contrato es lo que en la clase 008 convierte una corrida en evidencia.

🧮 Ejemplo trabajado

Reconstruyamos "por qué cambió la métrica" con el contrato completo. Estado A (reportado en un informe) y estado B (corrida de hoy):

Componente	Estado A	Estado B	¿Explica el cambio?
Commit	`a1b2c3d`	`a1b2c3d`	No (código idéntico)
Lockfile	numpy 1.26.4	numpy 2.1.0	**Candidato** (cambio mayor de versión)
Semilla	42	42	No
Datos	ventas_2025Q4.csv, sha256 `9f3e...`	mismo hash	No
Métrica	MAE = 12.3	MAE = 14.1	—

Diagnóstico en tres pasos reproducibles:

```
1. git diff A..B                      → vacío: el código no cambió
2. diff requirements-lock (A vs B)    → numpy 1.26.4 → 2.1.0
3. recrear venv con el lockfile de A  → MAE vuelve a 12.3  ■ causa aislada
```

Sin lockfile, este diagnóstico habría sido imposible: la diferencia se habría atribuido al modelo, a los datos o al azar. Nótese el método: cambiar **una** variable por vez, igual que en cualquier experimento.

📊 Propiedades y comparación

Herramienta	Qué fija	Qué NO fija	Costo de adopción
`venv` + lockfile	Versiones exactas de paquetes Python	Versión de Python, libs de sistema	Minutos
`pyenv` / instaladores	Versión del intérprete	Paquetes, SO	Minutos
Git (commit + tag)	Código y configuración versionada	Datos pesados, entorno	Horas de hábito
Semillas explícitas	Secuencia del PRNG	Validez estadística, no-determinismo GPU	Minutos
Contenedores (Docker)	SO + libs de sistema + Python + paquetes	Hardware, drivers	Días

⚠️ Errores conceptuales frecuentes

1. **"Fijé la semilla, mi resultado es válido."** La semilla da repetibilidad, no validez: si la conclusión cambia con otras semillas, lo reproducible era el ruido.
2. **"requirements.txt con rangos basta."** Los rangos resuelven distinto según la fecha de instalación; sin lockfile, dos máquinas "iguales" no lo son.
3. **"Git es una carpeta de backups."** Commits atómicos con mensajes explicativos y tags por experimento son metadatos científicos; un solo commit gigante "cambios" destruye la auditabilidad.
4. **"El notebook es el experimento."** Un notebook ejecutado fuera de orden con estado oculto no es reproducible; la lógica estable vive en módulos importables (como `ai_evolution.labs`) y el notebook solo orquesta y narra.
5. **"Versionar datos = meterlos en Git."** Git degrada con binarios grandes; lo correcto es registrar versión y checksum, y usar almacenamiento de datos dedicado.

🚀 Del aprendizaje a la operación

En equipos reales este contrato escala a: CI que reconstruye el entorno desde el lockfile y re-ejecuta los experimentos de humo en cada push; tracking de experimentos con herramientas dedicadas (MLflow, W&B) en lugar de hojas de cálculo; contenedores para fijar también el sistema operativo; y datos versionados con checksums verificados en el pipeline. La regla operativa no cambia: si un número no puede regenerarse desde commit + lockfile + semilla + datos, no entra en un informe.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("observability")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;

- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Documentación oficial de `venv` (Python) — uso: referencia consultada en su fuente original
- Documentación oficial de `random` — nota sobre semillas y determinismo — uso: referencia consultada en su fuente original
- Pro Git (Chacon & Straub) — libro oficial gratuito — uso: referencia consultada en su fuente original
- Sandve et al. (2013). Ten Simple Rules for Reproducible Computational Research — uso: fuente primaria del mecanismo estudiado
- Pineau et al. (2021). Improving Reproducibility in Machine Learning Research (checklist NeurIPS) — uso: fuente primaria del mecanismo estudiado

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P63 · Mejorar la reproducibilidad en la investigación en aprendizaje automático	2021	Convierte la reproducibilidad en un requisito operativo del proceso de publicación, con checklist, código y revisión.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 00 · capítulos de introducción y de agentes racionales

📝 Evaluación completa

❓ Preguntas

1. Define entornos python, git y experimentos reproducibles sin usar una marca o framework como definición.
2. Explica la relación entre Python, Git, semillas, entornos.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

010 — Cómo leer papers, benchmarks y claims de IA

Parte: 00 — Fundamentos, historia y método científico
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `evaluation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **cómo leer papers, benchmarks y claims de ia** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cómo leer papers, benchmarks y claims de ia usando los conceptos `papers`, `benchmarks`, `claims`, `replicación`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`papers`, `benchmarks`, `claims`, `replicación`

🗺️ Ubicación en el mapa de la IA

La IA produce decenas de miles de papers al año y un flujo constante de anuncios con métricas espectaculares; la capacidad de leerlos críticamente es tan parte del oficio como programar. Esta clase aplica la falsabilidad (clase 008) y la reproducibilidad (clase 009) al material publicado: cómo desarmar un paper, qué mide realmente un benchmark y qué preguntas hacerle a un claim antes de creerlo. Es la herramienta de supervivencia para el resto del programa, donde se leerán papers seminales en casi cada parte.

📖 Fundamentos

📄 Anatomía de un paper de ML

Las secciones tienen funciones retóricas distintas y se leen con niveles de confianza distintos:

- **Abstract/Introducción:** el claim en su versión más vendedora. Anotar qué se promete.
- **Método:** qué se hizo. Buscar los detalles que afectan la comparación: ¿mismo presupuesto de cómputo y de búsqueda de hiperparámetros para el baseline?

- **Experimentos:** la evidencia real. Aquí viven las tablas, los datasets y las condiciones. Es la sección contra la que se audita el abstract.
- **Limitaciones/Apéndices:** donde los autores honestos entierran las malas noticias (varianza entre semillas, resultados negativos, trucos de preprocesamiento).

Método de lectura en tres pasadas (Keshav): (1) título, abstract, figuras y conclusión — 5 minutos, decidir si sigue; (2) lectura completa sin demostraciones, anotando supuestos — 1 hora; (3) reconstrucción mental o real del experimento — la única pasada que habilita a citarlo como evidencia.

📏 Qué mide (y qué no) un benchmark

Un benchmark es una **muestra congelada de una tarea**, no la tarea. Cadena de representatividad que puede romperse en cada eslabón:

```
capacidad real → tarea idealizada → dataset recolectado → split de test → métrica agregada
```

Modos de fallo documentados:

- **Contaminación:** el test estuvo en los datos de entrenamiento (crítico con LLMs entrenados sobre la web: el benchmark público *está* en la web).
- **Sobreajuste comunitario (Goodhart):** años de optimizar contra el mismo leaderboard producen mejoras específicas del benchmark que no transfieren.
- **Artefactos y atajos:** el modelo explota regularidades espurias del dataset (longitud, léxico) en lugar de la capacidad nominal.
- **Saturación:** cerca del techo, las diferencias entre sistemas son ruido de anotación.
- **Métrica ≠ calidad:** accuracy agregada oculta el desempeño por subgrupos y el costo asimétrico de errores.

📑 Checklist para auditar un claim

Preguntas mínimas ante "X alcanza el estado del arte" o "X supera a los humanos":

```
1. ¿Comparación justa?    mismo dato, mismo cómputo, misma búsqueda de hiperparámetros
2. ¿Baselines fuertes?    ¿incluye el baseline trivial y el clásico bien afinado?
3. ¿Cuántas semillas?     ¿reportan media ± dispersión o una corrida afortunada?
4. ¿Población definida?   ¿"supera a humanos" — a cuáles, en qué subconjunto, con qué UI?
5. ¿Test limpio?          ¿auditaron contaminación/leakage? ¿test temporalmente posterior?
6. ¿Código y datos?       ¿se puede reproducir? ¿hay lockfile/pesos/configuración?
7. ¿Quién paga?           conflictos de interés declarados; los press releases no son papers
8. ¿Sobrevive fuera?      ¿hay evaluación out-of-distribution o replicación independiente?
```

Dodge et al. (2019) mostraron que reportar resultados sin el presupuesto de búsqueda de hiperparámetros hace incomparables los sistemas: con más intentos, cualquier método "gana". Pineau et al. (2021) convirtieron esta clase de preguntas en la checklist oficial de NeurIPS.

🎭 Tipología de claims

- **Claim de capacidad:** "el modelo puede razonar" — exige definición operativa y test falsable, si no, es marketing.

- **Claim de métrica:** "94.3 en el benchmark Y" — verificable pero estrecho; preguntar por la cadena benchmark→capacidad.
- **Claim de superioridad:** "supera a GPT-x / a médicos" — auditar la comparación (versión, prompt, fecha, subconjunto).
- **Claim de tendencia:** "la escala resolverá Z" — extrapolación; pedir la curva con incertidumbre, recordar la clase 003.

🧮 Ejemplo trabajado

Claim publicado: "Nuestro modelo diagnostica neumonía mejor que radiólogos (AUC 0.94 vs 0.87)". Auditoría con la checklist:

#	Pregunta	Hallazgo en el paper (caso realista)	Veredicto
1	¿Comparación justa?	Radiólogos sin historia clínica ni imágenes previas; el modelo evaluado en su distribución	Comparación asimétrica
2	¿Baseline?	No compara con la regla clínica estándar	Falta baseline fuerte
4	¿Población?	Un solo hospital, un solo fabricante de equipos	Generalización no demostrada
5	¿Test limpio?	Split aleatorio por *imagen*, no por *paciente*: imágenes del mismo paciente en train y test	**Leakage**
8	¿Fuera de distribución?	Sin validación externa	Pendiente

Reescritura honesta del claim: "En imágenes del hospital H con equipos del fabricante F, con partición por imagen (no por paciente), el modelo obtiene AUC 0.94 frente a 0.87 de radiólogos evaluados sin contexto clínico". Así formulado, el propio autor vería los huecos. Este patrón (split por imagen, un centro, comparación asimétrica) no es hipotético: es la falla modal que Kapoor y Narayanan catalogaron en la literatura de ML aplicado.

📊 Propiedades y comparación

Fuente	Fiabilidad típica	Sesgo dominante	Uso correcto
Paper revisado por pares	Media-alta	Publicación (solo éxitos)	Evidencia, tras pasada 2-3
Preprint (arXiv)	Variable	Sin filtro; velocidad	Señal temprana, verificar
Leaderboard público	Media	Goodhart, contaminación	Comparar tendencias, no décimas
Blog corporativo / press release	Baja	Interés comercial	Hipótesis a verificar, nunca evidencia
Replicación independiente	Alta	Escasez	El patrón oro cuando existe

⚠️ Errores conceptuales frecuentes

1. **"Está publicado/revisado por pares, luego es cierto."** La revisión filtra errores groseros, no verifica reproducibilidad; la tasa de resultados no replicables en ML aplicado es sustancial (Kapoor & Narayanan, 2023).
2. **"Mejor número en el benchmark = mejor sistema para mi problema."** El benchmark es una muestra congelada de otra distribución; la transferencia al dominio propio se mide, no se supone.
3. **"Superhumano en el test = superhumano en la tarea."** Los "humanos" del claim suelen ser anotadores con tiempo limitado y sin contexto; la tarea real incluye información y responsabilidad que el test excluye.
4. **"Una décima más de métrica importa."** Sin varianza entre semillas ni test de significancia, décimas son ruido; exigir media ± desviación sobre varias corridas.
5. **"El abstract resume fielmente el paper."** El abstract es la sección de ventas; la auditoría se hace contra la sección de experimentos y los apéndices.

🚀 Del aprendizaje a la operación

En la práctica profesional, este material se convierte en un protocolo de adopción de tecnología: ningún modelo o técnica entra al stack sin (a) replicar el claim clave en un subconjunto de datos propios, (b) comparar contra el baseline interno actual con el mismo presupuesto de ajuste, (c) registrar el experimento con el contrato de la clase 009, y (d) documentar la brecha entre el número publicado y el observado — que existirá, y cuya magnitud es información de planificación, no una decepción.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("evaluation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;

- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

📜 Dónde practicar esto

Este programa tiene un eje entero para aplicar lo de esta clase sobre papers reales: **148 fichas fundacionales**, de Pearson (1901) a DeepSeek-R1 (2025), cada una con su fuente primaria, sus límites y sus errores comunes documentados.

Recurso	Para qué sirve aquí
Cómo leer un paper de IA	El método aplicado al vocabulario y las convenciones del campo
Método de lectura en 5 pasadas	La extensión del método de Keshav que usa el eje
Fuentes y venues	Dónde buscar y cómo pesar arXiv frente a una conferencia revisada
Índice de papers	Los 148, por año y por bloque temático

Empieza por una ficha corta —P01 Perceptrón— y compara **tu** lectura del artículo original con lo que la ficha declara como fecha de consulta, evidencia y límites. Ese contraste es el ejercicio.

🔗 Referencias

- Keshav, S. (2007). How to Read a Paper (método de tres pasadas) — uso: fuente primaria del mecanismo estudiado
- Dodge et al. (2019). Show Your Work: Improved Reporting of Experimental Results — uso: fuente primaria del mecanismo estudiado
- Pineau et al. (2021). Improving Reproducibility in ML Research (checklist NeurIPS) — uso: fuente primaria del mecanismo estudiado
- Kapoor, S. & Narayanan, A. (2023). Leakage and the Reproducibility Crisis in ML-based Science — uso: fuente primaria del mecanismo estudiado
- Ioannidis, J. (2005). Why Most Published Research Findings Are False — uso: fuente primaria del mecanismo estudiado

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P62 · La IA y el benchmark del todo en el mundo entero	2021	Traslada al campo el concepto de validez de constructo: un número alto no prueba la capacidad que el benchmark dice medir.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 00 · capítulos de introducción y de agentes racionales

📝 Evaluación completa

❓ Preguntas

1. Define cómo leer papers, benchmarks y claims de ia sin usar una marca o framework como definición.
2. Explica la relación entre papers, benchmarks, claims, replicación.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

011 — Ética desde el diseño y límites de automatización

Parte: 00 — Fundamentos, historia y método científico
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `safety` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **ética desde el diseño y límites de automatización** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ética desde el diseño y límites de automatización usando los conceptos `ética`, `impacto`, `límites`, `supervisión`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`ética`, `impacto`, `límites`, `supervisión`

🗺️ Ubicación en el mapa de la IA

La ética aparece en la primera parte del programa, y no al final, por una razón de ingeniería: las decisiones con carga ética (qué datos usar, qué métrica optimizar, qué se automatiza y qué no) se toman en el diseño, cuando cambiarlas cuesta poco — no en la auditoría posterior, cuando cuesta el proyecto entero. Esta clase conecta la medida de desempeño de los agentes (clase 004) y la honestidad experimental (clases 008-010) con sus consecuencias sobre personas, y anticipa los marcos regulatorios que gobiernan la parte de despliegue.

📖 Fundamentos

📐 Ética desde el diseño (ethics by design)

Principio: los valores se implementan en artefactos técnicos concretos, no en declaraciones. Traducciones directas:

Decisión de diseño	Carga ética implícita
Métrica a optimizar	Qué errores importan y a quién perjudican
Datos de entrenamiento	Qué poblaciones están representadas y cuáles no
Umbral de decisión	Trade-off explícito entre falsos positivos y negativos
Interfaz y defaults	Qué conducta del usuario se induce
Registro y trazas	Si el daño podrá investigarse y atribuirse

Un sistema de scoring crediticio entrenado con decisiones históricas *ya contiene* una postura ética (perpetuar el patrón histórico); no adoptarla conscientemente no la elimina, solo la deja sin dueño.

⚖️ Conceptos operativos

- **Sesgo (bias):** error sistemático que afecta de forma desigual a subgrupos. Entra por los datos (muestreo, etiquetas históricas), por la métrica (agregada sobre subgrupos desiguales) o por el despliegue (uso fuera de la población de diseño). Se *mide* por subgrupo — accuracy global puede ocultar disparidades severas.
- **Equidad (fairness):** existen múltiples definiciones formales (paridad demográfica, igualdad de oportunidades, calibración por grupo) y son **mutuamente incompatibles** en general (resultado de imposibilidad de Kleinberg et al. / Chouldechova): elegir una es una decisión de política, no un detalle técnico.
- **Transparencia y rendición de cuentas:** documentación estandarizada del modelo (*Model Cards*, Mitchell et al. 2019) y de los datos (*Datasheets for Datasets*, Gebru et al. 2021): uso previsto, poblaciones evaluadas, límites conocidos.
- **Supervisión humana significativa:** que el humano en el circuito pueda *realmente* discrepar del sistema (tiempo, información, autoridad e incentivos para hacerlo); un humano que aprueba en cadena lo que el sistema propone es teatro de supervisión (*automation bias* documentado).

🚦 Límites de automatización: qué no se delega

Criterios para decidir el grado de autonomía admisible de un sistema:

```
1. Reversibilidad del daño:   ¿el error se puede deshacer? (recomendación de película
                              vs. denegación de crédito vs. diagnóstico)
2. Costo asimétrico:          ¿un falso negativo vale lo mismo que un falso positivo?
3. Población vulnerable:      ¿el error cae sobre quien menos puede defenderse?
4. Explicabilidad exigible:   ¿la decisión debe poder justificarse individualmente
                              (derecho, medicina, empleo)?
5. Deriva del entorno:        ¿la distribución cambia más rápido de lo que se re-evalúa?
```

Cuanto más alto puntúa un caso en estos ejes, más cerca debe estar del extremo "el sistema solo *informa*, el humano decide" y más lejos de "el sistema decide y ejecuta". Los marcos regulatorios modernos codifican esta gradación: el AI Act europeo (Reglamento 2024/1689) clasifica sistemas por nivel de riesgo (inaceptable/alto/limitado/mínimo) con obligaciones proporcionales, y el NIST AI RMF estructura el ciclo govern-map-measure-manage.

📑 El artefacto de esta clase

El laboratorio `safety` ejercita la versión mínima: una decisión automatizada con umbral, la tabla de errores por subgrupo y la declaración explícita de `limitations`. El artefacto `risk_and_limitations.md` de cada clase del programa es un Model Card embrionario.

🧮 Ejemplo trabajado

Un clasificador decide qué solicitudes de crédito revisar manualmente. Métricas por subgrupo sobre 2 000 casos de test (1 000 por grupo), con el mismo umbral global 0.5:

	Grupo A	**Grupo B**
Tasa real de impago	10 %	10 %
Recall (impagos detectados)	80/100 = 0.80	50/100 = 0.50
Falsos positivos (buenos pagadores rechazados)	90/900 = 0.10	180/900 = 0.20
Accuracy del grupo	(80+810)/1000 = **0.89**	(50+720)/1000 = **0.77**
Accuracy global reportada	(890+770)/2000 = **0.83**	

El informe ejecutivo diría "accuracy 0.83". La tabla por subgrupo muestra que el grupo B sufre el doble de rechazos injustos (0.20 vs 0.10) y la mitad de protección frente a impagos reales (0.50 vs 0.80) **con la misma tasa base**: el modelo está peor calibrado para B (típicamente por menos datos históricos de ese grupo). Opciones de diseño, cada una con carga ética explícita: recolectar datos de B, umbrales por grupo (¿es legal en la jurisdicción?), o enviar B a revisión humana (¿con qué sesgos revisa el humano?). No hay opción neutra; la ética desde el diseño consiste en elegir con los números sobre la mesa y documentar la elección.

📊 Propiedades y comparación

Enfoque	**Cuándo actúa**	**Costo de corregir**	**Ejemplo de artefacto**
Ética desde el diseño	Antes de construir	Bajo	Métrica por subgrupo elegida ex ante, datasheet
Auditoría posterior	Tras construir	Alto	Informe de sesgo sobre sistema en uso
Cumplimiento reactivo	Tras el incidente	Máximo (+daño)	Retirada del sistema, sanción
Teatro ético	Nunca (solo comunica)	—	Principios sin métricas ni dueños

⚠️ Errores conceptuales frecuentes

1. **"La ética es subjetiva, no se puede medir."** Las consecuencias se miden: tasas de error por subgrupo, calibración por grupo, daños reportados. Lo valorativo es elegir el trade-off; lo técnico es exponerlo con números.
2. **"Quitamos la variable protegida, ya no hay sesgo."** Las variables correlacionadas (código postal, historial) la reconstruyen (*proxy discrimination*); la equidad se evalúa sobre resultados, no sobre columnas eliminadas.
3. **"Un humano revisa, así que hay supervisión."** Sin tiempo, información e incentivos para discrepar, el humano ratifica (automation bias); la supervisión se diseña y se mide (tasa de discrepancia) como cualquier componente.
4. **"Optimizar todas las definiciones de fairness a la vez."** Es matemáticamente imposible en general; hay que elegir la definición apropiada al dominio y justificarla.
5. **"La regulación mata la innovación, mejor ignorarla hasta el final."** Los requisitos del AI Act y NIST AI RMF (documentación, gestión de riesgo, datos de calidad) son práctica de ingeniería sólida; adoptarlos tarde cuesta rediseños completos.

🚀 Del aprendizaje a la operación

En producción, esta clase se convierte en artefactos con dueño: un registro de decisiones de diseño con su justificación ética (qué métrica, qué umbral, por qué); model cards y datasheets versionados junto al código; evaluación por subgrupo en el pipeline de CI, no solo en el informe inicial; un canal de reporte de daños con SLA de respuesta; y un mapa de qué decisiones el sistema tiene prohibido tomar solo — revisado cuando cambia la población, el modelo o la ley.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("safety")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Mitchell et al. (2019). Model Cards for Model Reporting — uso: fuente primaria del mecanismo estudiado
- Gebru et al. (2021). Datasheets for Datasets — uso: fuente primaria del mecanismo estudiado
- NIST AI Risk Management Framework — uso: marco normativo de referencia
- Reglamento (UE) 2024/1689 — AI Act (texto oficial) — uso: marco normativo de referencia
- Kleinberg, Mullainathan & Raghavan (2016). Inherent Trade-Offs in the Fair Determination of Risk Scores — uso: fuente primaria del mecanismo estudiado
- ACM Code of Ethics and Professional Conduct — uso: referencia consultada en su fuente original

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P61 · Sobre los peligros de los loros estocásticos: ¿pueden ser demasiado grandes los modelos de lenguaje?	2021	Pone por escrito el coste de la carrera por el tamaño: quién paga, quién queda representado y qué se afirma de más sobre la comprensión.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 00 · capítulos de introducción y de agentes racionales

Normas y documentación oficial que aplica esta clase: AI Risk Management Framework · Reglamento (UE) 2024/1689

📝 Evaluación completa

❓ Preguntas

1. Define ética desde el diseño y límites de automatización sin usar una marca o framework como definición.
2. Explica la relación entre ética, impacto, límites, supervisión.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

012 — Proyecto: mapa evolutivo verificable de la IA

Parte: 00 — Fundamentos, historia y método científico
Nivel: fundamentos · **Horas estimadas:** 10
Laboratorio: `capstone` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **proyecto: mapa evolutivo verificable de la ia** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: mapa evolutivo verificable de la ia usando los conceptos `timeline`, `evidencia`, `fuentes`, `taxonomía`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`timeline`, `evidencia`, `fuentes`, `taxonomía`

🗺️ Ubicación en el mapa de la IA

Esta clase cierra la parte 00 integrando sus once piezas en un artefacto único: una línea de tiempo de la IA donde **cada hito es una afirmación verificable con fuente primaria**, clasificada en una taxonomía explícita y acompañada de su evidencia y sus límites. No es un ejercicio de memoria histórica: es el primer proyecto donde el estudiante aplica el método del programa completo (claim → fuente → evidencia → limitación) a un corpus real, el mismo método que usará para evaluar modelos y papers en las partes siguientes.

📖 Fundamentos

🗂️ Qué es un hito verificable

Un hito del mapa no es "en 1997 la IA venció al ajedrez", sino un registro estructurado:

```
hito:
  fecha:        1997-05-11
  afirmación:   "Deep Blue (IBM) vence a Kasparov 3.5-2.5 en match a 6 partidas"
  taxonomía:    [búsqueda, juegos, hardware especializado]
  fuente_primaria: paper/documento del equipo o registro del evento
  evidencia:    resultado del match, documentado por múltiples fuentes independientes
  limitación:   dominio cerrado; sin transferencia; hardware ad hoc (norma de la clase 001:
                la conclusión no excede los datos)
```

Los criterios de calidad vienen de las clases anteriores: la afirmación debe ser falsable (clase 008), la fuente debe ser primaria o canónica, no un press release (clase 010), y la clasificación debe usar la taxonomía del campo (clases 001-004).

🌳 Una taxonomía operativa para clasificar hitos

Tres ejes de clasificación que el proyecto debe aplicar de forma consistente:

1. **Paradigma:** simbólico/lógico (Logic Theorist, sistemas expertos) · conexionista (perceptrón, deep learning) · probabilístico/estadístico (redes bayesianas, ML clásico) · híbrido/agéntico.
2. **Tipo de avance:** teórico (indecidibilidad, NP-completitud) · algorítmico (retropropagación, atención) · de sistema/ingeniería (Deep Blue, AlphaGo) · de datos o hardware (ImageNet, GPU) · institucional (Dartmouth, informes ALPAC/Lighthill).
3. **Estado de la evidencia:** verificado con fuente primaria · verificado con fuentes secundarias sólidas · claim de la época luego refutado o matizado (¡también son hitos!: las promesas de Simon de 1965 pertenecen al mapa *como claims fallidos*).

Registrar claims fallidos con la misma disciplina que los éxitos es lo que hace al mapa *evolutivo* y no hagiográfico: los inviernos (clase 003) solo se entienden si el mapa conserva las promesas que los precedieron.

🔍 Protocolo de verificación de cada hito

```
1. Redactar la afirmación en forma falsable (fecha, actor, resultado medible).
2. Localizar la fuente primaria (paper con DOI, informe oficial, propuesta original).
3. Contrastar 1 fuente secundaria independiente (AIMA cap. 1, Nilsson).
4. Anotar qué NO demuestra el hito (limitación honesta).
5. Clasificar en los 3 ejes de la taxonomía.
6. Registrar el hito en el formato estructurado (JSON/YAML) del proyecto.
```

El laboratorio `capstone` valida mecánicamente la parte estructural (campos presentes, fechas coherentes, taxonomía cerrada); la calidad de las fuentes la valida la rúbrica de `assessment.md` — un validador automático no puede saber si una fuente es primaria, y esa distinción entre lo verificable por máquina y lo que exige juicio es en sí misma una lección del programa.

🧭 Esqueleto mínimo del mapa (12 hitos ancla)

El proyecto parte de un esqueleto que el estudiante debe verificar y extender: 1936 (Turing, computabilidad) · 1943 (McCulloch-Pitts) · 1950 (juego de imitación) · 1956 (Dartmouth) · 1965 (promesa de Simon, *claim fallido*) · 1969 (Perceptrons) · 1973 (Lighthill) · 1986 (retropropagación popularizada) · 1997 (Deep Blue) · 2012 (AlexNet) · 2017 (atención/transformers) · 2022+ (LLMs desplegados masivamente). Cada uno debe pasar el protocolo de 6 pasos; el estudiante añade al menos 8 hitos propios con el mismo estándar.

🧮 Ejemplo trabajado

Apliquemos el protocolo completo al hito de 1950:

Paso	Resultado
1. Afirmación falsable	"En octubre de 1950, A. M. Turing publica en *Mind* LIX(236) el artículo que propone el juego de imitación como criterio operativo"
2. Fuente primaria	DOI 10.1093/mind/LIX.236.433 (verificable: resuelve al artículo)
3. Fuente secundaria	AIMA 4e, §1.1; Nilsson (2010), cap. 2
4. Limitación honesta	El artículo NO afirma que las máquinas piensen ni fija el test como definición de IA; propone un sustituto operativo y predice (falsablemente) el desempeño hacia el año 2000
5. Taxonomía	Paradigma: pre-paradigmático/fundacional · Tipo: teórico · Evidencia: primaria verificada
6. Registro	`{"date":"1950-10","claim":"...","source_doi":"10.1093/mind/LIX.236.433","taxonomy":["foundational","theoretical"],"limitations":["no define IA","criterio conductual, no de competencia"]}`

Contraejemplo instructivo: la cifra popular "Turing predijo que en 2000 las máquinas pasarían el test" es imprecisa — lo que el artículo dice es que hacia el año 2000 un interrogador promedio tendría no más de 70 % de probabilidad de identificar correctamente tras 5 minutos. La versión popular es *menos falsable* que la original: reconstruir el claim exacto desde la fuente es el músculo que este proyecto entrena.

📊 Propiedades y comparación

Enfoque de línea de tiempo	Fuentes	Claims fallidos	¿Auditable?	Valor formativo
Divulgativa (blog, marketing)	Secundarias o ninguna	Omitidos	No	Bajo: narrativa de progreso lineal
Académica narrativa (AIMA §1.1)	Mixtas, citadas	Mencionados	Parcial	Alto, pero no estructurado
Mapa verificable (este proyecto)	Primarias + contraste	Registrados como hitos	Sí (formato + protocolo)	Alto y reutilizable programáticamente

⚠️ Errores conceptuales frecuentes

1. **"Una línea de tiempo es neutral."** Toda selección de hitos es una tesis sobre qué importa; la taxonomía explícita y los claims fallidos incluidos hacen la tesis auditable en lugar de implícita.
2. **"Wikipedia/un blog es fuente suficiente."** Son puntos de partida para *localizar* la fuente primaria, no la fuente; el protocolo exige llegar al documento original o a una referencia canónica con página/capítulo.
3. **"Los fracasos no son hitos."** ALPAC, Lighthill y las promesas incumplidas explican la forma de la curva tanto como los éxitos; omitirlos produce el sesgo de supervivencia de la clase 008.
4. **"Si el validador pasa, el proyecto está bien."** El validador comprueba estructura; la calidad de fuentes y la honestidad de las limitaciones las evalúa la rúbrica. Confundir validación mecánica con validez es exactamente el error de la clase 010 con benchmarks.
5. **"Más hitos = mejor mapa."** Veinte hitos verificados con protocolo completo valen más que cien copiados; la densidad sin verificación reintroduce el ruido que el proyecto busca eliminar.

🚀 Del aprendizaje a la operación

El patrón de este proyecto — registro estructurado + fuente + evidencia + limitación + validación automática de estructura + revisión humana de calidad — es el mismo que en la industria toman los *model cards*, los registros de decisiones de arquitectura (ADR) y los inventarios de sistemas de IA que exigen marcos como el AI Act. Quien sabe construir un mapa histórico auditable sabe construir el inventario auditable de los sistemas de su organización: cambia el contenido, no el método.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("capstone")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

Referencias

- Turing, A. M. (1950). Computing Machinery and Intelligence. *Mind*, LIX(236) — uso: fuente primaria del mecanismo estudiado
- McCarthy, Minsky, Rochester & Shannon (1955). Propuesta de Dartmouth (documento original) — uso: referencia consultada en su fuente original
- Lighthill, J. (1973). Artificial Intelligence: A General Survey — uso: referencia consultada en su fuente original
- Russell, S. & Norvig, P. *AIMA*, 4.ª ed., §1.1 (historia, para contraste secundario) — uso: desarrollo extendido del tema

- Nilsson, N. (2010). *The Quest for Artificial Intelligence* (PDF oficial gratuito) — uso: referencia consultada en su fuente original

---

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 00

📝 Evaluación completa

❓ Preguntas

1. Define proyecto: mapa evolutivo verificable de la ia sin usar una marca o framework como definición.
2. Explica la relación entre timeline, evidencia, fuentes, taxonomía.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.